

Stellenbosch
UNIVERSITY
IYUNIVESITHI
UNIVERSITEIT

Backseat driving: Using privileged-information auxiliary losses in autonomous racing

by

Jacques Friedrich Amsel

Report submitted in partial fulfillment of the requirements of the module Project (E) 448 for the degree Baccalaureus in Engineering in the Department of Electrical and Electronic Engineering at the University of Stellenbosch

Study leader: Dr. JC Schoeman

October 2026

Acknowledgements

This work would not have been possible without the mentorship of my supervisor, Dr. JC Schoeman, who provided valuable guidance and feedback throughout the project. I would also like to thank my friends and peers for their encouragement and support, as well as my family for their support throughout my studies. Finally, I would like to thank the developers of the open-source software and libraries that were used in this project, including, but not limited to, PyTorch, NumPy, Matplotlib, Seaborn, Pandas, Python, and Ubuntu. Without their contributions, this work would have been immensely more difficult.

"Now, this is podracing!"

- *Anakin Skywalker in Star Wars: The Phantom Menace*

Plagiarism declaration

1. Plagiaat is die oorneem en gebruik van die idees, materiaal en ander intellektuele eiendom van ander persone asof dit jou eie werk is. *Plagiarism is the use of ideas, material and other intellectual property of another's work and to present it as my own.*
2. Ek erken dat die pleeg van plagiaat 'n strafbare oortreding is aangesien dit 'n vorm van diefstal is. *I agree that plagiarism is a punishable offence because it constitutes theft.*
3. Dienooreenkomstig is alle aanhalings en bydraes vanuit enige bron (ingesluit die internet) volledig verwys (erken). Ek erken dat die woordelikse aanhaal van teks sonder aanhalingstekens (selfs al word die bron volledig erken) plagiaat is. *Accordingly all quotations and contributions from any source whatsoever (including the internet) have been cited fully. I understand that the reproduction of text without quotation marks (even when the source is cited) is plagiarism.*
4. Ek verklaar dat die werk in hierdie skryfstuk vervat, behalwe waar anders aangedui, my eie oorspronklike werk is en dat ek dit nie vantevore in die geheel of gedeeltelik ingehandig het vir bepunting in hierdie module/werkstuk of 'n ander module/werkstuk nie. *I declare that the work contained in this assignment, except where otherwise stated, is my original work and that I have not previously (in its entirety or in part) submitted it for grading in this module/assignment or another module/assignment.*

26938839 Student number	 Signature
JF Amsel Initials and surname	29 October 2026 Date

Abstract

End-to-end reinforcement learning (RL) agents can drive a car directly from raw sensor data. However, an agent that drives from LiDAR alone must infer learn quantities it cannot observe, such as its own velocity, its tyre slip, and the curvature of the track ahead. In simulation, these quantities are available as privileged information, but the reward gives only an indirect, high-variance signal for learning them. This project investigates whether auxiliary losses for predicting privileged information from the policy's shared representation can improve the training speed, racing performance, and internal representations of a LiDAR-only racing agent.

A custom racing simulator was built in Unity, with dynamics calibrated to the RoboRacer platform, and a procedural track generator. Both feedforward and recurrent agents were trained with Proximal Policy Optimisation (PPO), and linear probes were used to measure which privileged quantities their learned representations encode. Ten categories of auxiliary targets were each evaluated at three loss weights across 6 seeds: velocities, yaw rate, track curvature at three lookahead distances, and per-wheel slip.

Probing shows that the baseline agents already encode the vehicle's instantaneous dynamics at initialisation, and that training trades this information away in favour of upcoming track curvature. Close-ahead track curvature (up to 3m ahead) is the only auxiliary target that meaningfully benefits both architectures. Most other targets degrade the feedforward agent, while the recurrent agent benefits from many of them, which suggests that auxiliary losses chiefly ease the training of the harder-to-train architecture. Finally, making a quantity more decodable does not always improve driving, and the effect of each target depends strongly on its loss weight.

Table of contents

Acknowledgements **i**

Plagiarism declaration **iii**

Abstract **iv**

Notation **viii**
Conventions . . . viii
Reference . . . viii

Acronyms and abbreviations **xi**

1. Introduction **1**
1.1. Background and motivation . . . 1
1.2. Problem Statement and Objectives . . . 1
1.3. Solution Overview and Scope . . . 2
1.4. Report Outline . . . 2

2. Background and Related Literature **4**
2.1. Previous Approaches . . . 4
2.1.1. Classical approaches . . . 4
2.1.2. Learning approaches . . . 4
2.2. Learning Methods Using Privileged Information . . . 5
2.2.1. Asymmetric Actor-critic Methods . . . 5
2.2.2. Teacher-student distillation . . . 5
2.2.3. Auxiliary learning . . . 6
2.2.3.1. Auxiliary Rewards . . . 6
2.2.3.2. Auxiliary losses . . . 6
2.3. Evaluation of existing approaches . . . 7

3. System overview **8**

4. Simulator **9**
4.1. Simulator performance comparison . . . 10
4.2. Unity Vehicle Model . . . 11
4.2.1. The RigidBody . . . 11
4.2.2. The WheelCollider . . . 11
4.2.3. Steering . . . 13
4.2.4. LiDAR . . . 13

4.3. The track . . . 13
4.3.1. Centerline generation . . . 13
4.3.2. Track walls . . . 15
4.3.3. Track Curvature . . . 16

5. The Agent **17**
5.1. Partially Observable Markov Decision Processes (POMDPs) . . . 17
5.1.1. Value functions . . . 19
5.2. Reinforcement Learning (RL) . . . 19
5.2.1. Policy gradient theorem . . . 19
5.2.2. REINFORCE . . . 20
5.2.3. Proximal Policy Optimisation (PPO) . . . 20
5.3. Neural Network Architecture . . . 21
5.3.1. Policy Head . . . 22
5.3.2. Auxiliary heads . . . 23
5.3.3. Choice of Auxiliary Loss Function . . . 24
5.4. Finding the Optimal Weights . . . 25
5.4.1. Hyperparameter Tuning . . . 26
5.4.2. Adaptive Weighting . . . 27
5.4.3. Weighting Scheme Used in This Project . . . 27
5.5. Metrics . . . 28
5.5.1. Progress . . . 28
5.5.2. Collision Rates . . . 28
5.5.3. Reward function . . . 29
5.5.4. Area Under the Curve (AUC) . . . 29
5.5.5. R^2 score . . . 29
5.6. Trainer Performance Tuning . . . 30

6. Experiments & results **32**
6.1. Baseline Model Performance . . . 32
6.2. Learned Features of the Baseline Models . . . 32
6.3. Agent Performance After Auxiliary Learning . . . 34
6.4. Learned Features After Auxiliary Learning . . . 38

7. Conclusions and recommendations **40**
7.1. Summary . . . 40
7.2. Future work . . . 40

Bibliography **41**

Appendix **48**
Appendix A Project Timeline . . . 49
Appendix B Graduate Attributes . . . 50
B.1 Complex Engineering Problem . . . 50
Appendix C AI Declaration . . . 51
Appendix D WheelCollider properties . . . 52

Appendix E Derivation of the Policy Gradient Theorem 53
Appendix F Distribution of Auxiliary Targets 55
Appendix G Baseline R^2 Scores 56
Appendix H Mean Reward Per Timestep of a Baseline FF Model 57
Appendix I Training Performance Benchmark Results 58
Appendix J R^2 Scores After Training with Auxiliary Heads 59

Notation

Conventions

The symbols in this thesis follow a small set of conventions, so a new expression can usually be read without looking it up.

- **Capital vs. lowercase.** A capital letter is a random variable (S_t, A_t, O_t), and the matching lowercase letter is a particular realized value of it (s_t, a_t, o_t).
- **Calligraphic letters** denote sets or spaces: $\mathcal{S}$, $\mathcal{O}$, $\mathcal{A}$ are the state, observation, and action spaces. Exceptions are the reward function $\mathcal{R} : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to \mathbb{R}$, the observation function, $\mathcal{Z} : \mathcal{S} \to \Delta_{\mathcal{O}}$, and the transition function $\mathcal{P} : \mathcal{S} \times \mathcal{A} \to \Delta_{\mathcal{S}}$, which are not sets or spaces. Lowercase symbols denote quantities that live in the corresponding space, e.g. $s \in \mathcal{S}$.
- **Subscript t** indexes time, so x_t is the quantity x at time step t.
- **Subscript θ** marks dependence on learnable parameters, e.g. π_θ.
- **Superscript π** means "under policy π", e.g. V^π, Q^π, A^π.
- **Hat** marks an estimate: $\hat{X}$ estimates the random variable X, and $\hat{x}$ is a sample estimate of it. $\hat{P}(X = x)$ is a sample estimate of the probability that $X = x$.
- **Tilde** means "distributed as": $X \sim Y$ is exact, and $X \dot{\sim} Y$ is approximate.
- **Conditioning bar.** In $f(a \mid b)$, quantities after the bar are given. This holds for both probabilities and expectations.
- **Expectation subscripts** say what the expectation is over: $\mathbb{E}_\pi$ is over trajectories generated by policy π, and $\mathbb{E}_{x \sim p}$ is over x drawn from distribution p.

Reference

Reinforcement learning setup

$\mathcal{S}$	Set of all states
$\mathcal{O}$	Observation space
$\mathcal{A}$	Action space
S_t, s_t	State at time t (random variable, realization)
O_t, o_t	Agent's observation at time t (random variable, realization)

A_t, a_t	Action taken at time t (random variable, realization)
r_t	Reward received at time t
$\rho_0(s)$	Initial state distribution
γ	Discount rate
λ	GAE discount rate

Policies and value functions

π_θ	Policy with parameters θ
$\pi_\theta(a_t \mid o_t)$	Probability density of action a_t given observation o_t under policy parameters θ
$G_t = \sum_{i=t}^{T} \gamma^{i-t} \cdot r_i$	Discounted return at time t
$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$	Value function for policy π at state s
$Q^\pi(s,a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$	Action-value function for policy π at state s and action a
$A^\pi(s,a) = Q^\pi(s,a) - V^\pi(s)$	Advantage function for policy π at state s and action a

Calculus

$\nabla_\theta f(\theta)$	Gradient of the function $f(\theta)$ with respect to parameters θ

Probability and statistics

$\Delta_\mathcal{Y}$	Set of all probability distributions with support $\mathcal{Y}$
$X \sim Y$	X is distributed according to Y
$X \dot{\sim} Y$	X is approximately distributed according to Y
$\mathbb{E}[X]$	Expected value of X
$\mathbb{E}_{x\sim p}[f(x)]$	Expected value of $f(x)$ when x is distributed according to p
$\mathrm{Var}(X)$	Variance of X
$N(\mu, \sigma^2)$	Gaussian distribution with mean μ and variance σ^2

$\text{Uniform}(a, b)$	Uniform distribution with support $[a, b]$, evaluated at x
$\hat{X}$	Estimator for X
$\hat{x}$	Sample estimate of X

Acronyms and abbreviations

AUC	Area under the curve
C.I	Confidence interval
CNN	Convolutional neural network
FOV	Field of view
GAE	Generalised advantage estimation
H2H	Head-to-head
IMU	Inertial measurement unit
LiDAR	Light detection and ranging
MDP	Markov decision process
MAE	Mean absolute error
MSE	Mean squared error
POMDP	Partially observable Markov decision process
PPO	Proximal policy optimisation
RNN	Recurrent neural network
RL	Reinforcement learning
RMSE	Root mean squared error
SE	Standard error

Chapter 1

Introduction

1.1. Background and motivation

Road traffic crashes remain a leading cause of death and serious injury worldwide [1], and the overwhelming majority are attributed to human error [2], as cited by Medina et al. [3]. Autonomous driving promises to reduce severe and fatal accidents [4], improve traffic flow [5], and reduce emissions [6]. Before such systems can be trusted on public roads, however, they must handle difficult situations, where small mistakes compound quickly. Autonomous racing is a demanding proxy for these situations because high speeds, tight control margins, and short recovery times stress the driving policy.

Broadly, autonomous driving control architectures fall into two paradigms: modular pipelines and end-to-end learning. Modular pipelines decompose the driving task into distinct sub-problems, such as perception, planning, and control; this provides interpretability and simplifies debugging, but often demands extensive domain expertise and manual tuning. End-to-end learning instead maps raw sensor inputs (e.g., LiDAR or camera frames) directly to control outputs (e.g., speed and steering angle). It avoids hand-engineered intermediate representations, but typically requires vast amounts of data and suffers from poor interpretability, making failures difficult to diagnose [7].

Auxiliary learning, as popularised by Jaderberg et al. [8], presents a compelling method to address these shortcomings. By training a model to perform secondary tasks alongside its primary control objective, the network is encouraged to learn richer, more structured intermediate representations [9] while improving interpretability [7]. In simulation, privileged ground-truth state is available during training even when it is unavailable to the deployed policy. This motivates the question of whether such auxiliary signals can improve an end-to-end racing agent that must drive from limited onboard sensing.

1.2. Problem Statement and Objectives

An end-to-end reinforcement learning (RL) agent that drives from LiDAR alone has to learn representations of privileged information that it cannot directly observe in the real world: its own velocity and slip, upcoming track curvature, and, in Head-to-Head (H2H) racing, the opponent's location. Early in training, the agent has not yet learned

to drive well enough to explore the track effectively, so the reward signal that would teach it these representations is sparse and noisy. This can slow convergence and lead to sub-optimal policies. Privileged information is, however, available in simulation.

The objective of this report is to investigate whether *auxiliary heads* that predict privileged information from the policy's shared representation can improve a LiDAR-only end-to-end racing agent. The objectives are to:

(a) select an appropriate simulation environment for autonomous racing;
(b) implement a baseline end-to-end RL agent that drives from LiDAR alone;
(c) quantify what representations the baseline agent natively encodes;
(d) implement auxiliary learning to provide additional supervisory signals to the agent during training; and
(e) compare the auxiliary-learning agent against the baseline on driving performance and representation quality.

1.3. Solution Overview and Scope

The project was executed in three phases: simulation development, baseline evaluation, and auxiliary learning experimentation. First, a custom 3D racing simulator was built in Unity using PhysX-based vehicle dynamics and a procedural track generator. This custom simulator was benchmarked against the standard F1Tenth Gym [10]. Second, a baseline end-to-end reinforcement learning (RL) agent was trained with Proximal Policy Optimisation (PPO) and a dense progress-based reward. Third, auxiliary heads were added to the network to predict privileged state information during training. The motivation for this approach is developed in the following chapters.

The scope is limited to simulation; no sim-to-real transfer to a physical RoboRacer was attempted. The agent drives from LiDAR and its current steering and throttle only, using either a memoryless feedforward policy, or a recurrent policy. Learning is done using PPO, and auxiliary losses are studied; auxiliary rewards and reward shaping are discussed as background but not evaluated. Only single-agent racing is studied, so opponent information is not considered.

1.4. Report Outline

Chapter 2 reviews the background and related literature. It contrasts classical modular pipelines with end-to-end learning approaches to autonomous racing, and surveys how privileged information has previously been exploited through other methods. The chapter closes by identifying the gap in the literature that this report addresses: auxiliary losses have not been studied for LiDAR-only end-to-end racing agents, and it is unknown which privileged targets help or how they reshape the policy's representation.

Chapter 3 provides a brief system overview, describing how the simulator and Python training script communicate, and how observations, actions, and rewards flow between them.

Chapter 4 presents the custom Unity simulator. It justifies the choice of simulator, benchmarking it against the F1Tenth gym, and describes the vehicle dynamics implemented, as well as the procedural track generation and privileged information available during training.

Chapter 5 describes the agent and auxiliary learning method. It formalises racing as a POMDP, introduces PPO and the network architecture, and defines the auxiliary targets, loss functions, and weighting strategy, together with the metrics used for evaluation.

Chapter 6 reports the experiments performed and their results. It first characterises the baseline agent's performance, and, using linear probes, what its internal representation natively encodes; it then evaluates each auxiliary configuration against the baseline on driving performance and representation quality. This chapter tests the central claim that auxiliary losses improve both representation learning and racing behaviour.

Chapter 7 concludes the report. It summarises the findings against the objectives, discusses limitations, and outlines future work, thereby placing our contributions in the context of the broader autonomous racing literature.

Chapter 2

Background and Related Literature

Before we can motivate the use of auxiliary losses to exploit privileged information to improve a LiDAR-only end-to-end racing agent, we first review the literature on autonomous racing. We contrast classical modular pipelines with end-to-end learning approaches, and survey how privileged information has previously been exploited through other methods. This chapter closes by identifying the gap in the literature that this report addresses: auxiliary losses have not been studied for LiDAR-only end-to-end racing agents, and it is unknown which privileged targets help or how they reshape the policy's representation.

2.1. Previous Approaches

Evans et al. [11] present a survey breaking down the autonomous racing literature in RoboRacer into two categories: *classical approaches* and *learning approaches*. We shall provide a brief overview of classical approaches, and then focus on learning approaches, which are more relevant to this project.

2.1.1. Classical approaches

Classical approaches generally decompose the driving task into three stages: perception, planning, and control [11]. Perception is the process of converting raw sensor inputs into a representation of the environment, such as a map of the track and the location of other vehicles. Planning uses this representation to generate a trajectory for the vehicle to follow, and control executes this trajectory by sending commands to the vehicle's actuators. These approaches tend to be more interpretable and easier to debug [12], as each component can be analysed separately [13].

2.1.2. Learning approaches

Learning approaches, which have recently shown promise in the field of autonomous driving in simulation [14], [15] as well as in deployment on a physical vehicle [16], are often end-to-end, mapping raw sensor inputs directly to control outputs [13]. This can be done using supervised learning (such as behaviour cloning [17]), where the model is trained on a dataset of expert demonstrations, or using reinforcement learning [18],

where the model learns by interacting with the environment and receiving feedback in the form of rewards.

Learning approaches have the potential to learn complex behaviours that are difficult to hand-engineer, and can adapt to changing environments [19]. However, they often require large amounts of data and computational resources [20], and can be less interpretable than classical approaches; deep end-to-end learning approaches are often regarded as "black boxes" [21], which may be undesirable in safety-critical applications [22] like driving. In RoboRacer, learning approaches have shown promise, but were unable to outperform classical approaches in benchmarks [11], and recently the majority of 27th RoboRacer Autonomous Racing Competition entries were classical approaches [23], despite learning approaches generalising better to unseen tracks [19].

2.2. Learning Methods Using Privileged Information

During training in simulation, information that is not available to the agent at inference time is available. This information is referred to as *privileged information*, and is usually unused. There are, however, several methods that use privileged information to improve the performance of an agent.

Common ways to use privileged information include as inputs to training-only components (e.g., an asymmetric critic), as inputs to a teacher for distillation, and as auxiliary rewards or losses. We provide these examples to illustrate how other methods use privileged information, and to motivate the use of auxiliary losses as an alternative method for autonomous racing.

2.2.1. Asymmetric Actor-critic Methods

Asymmetric actor-critic methods [24] are a variant of actor-critic methods that train two distinct models: one for the policy, which chooses the agent's actions and only receives the agent's observations as input, and one for the critic, which receives additional privileged information as input. The critic is used to estimate the value of each state, which is then used to update the agent to choose actions that lead it into more valuable states. Vasco et al. [25] show that an asymmetric critic can achieve better-than-human performance using a vision-based RL agent in a simulated autonomous racing environment, and perform better than a baseline symmetric actor-critic agent.

2.2.2. Teacher-student distillation

Teacher-student distillation [26] involves training a teacher model with access to privileged information, and then using the teacher's outputs to train a student model that does not have access to this information. The student model is trained to mimic the teacher's behaviour, effectively learning from the teacher's understanding of its privileged knowledge. Xu et al. [27] applied this method to LiDAR-based autonomous

racing on the RoboRacer platform, and found that the student model was able to achieve better performance than a baseline model trained without a teacher.

2.2.3. Auxiliary learning

Auxiliary learning is a method that falls under the umbrella of multi-task learning, where a model is trained to perform multiple tasks simultaneously. Specifically, in auxiliary learning, the model's performance on the auxiliary task is not an objective in itself, but the model's performance on the auxiliary task is used to improve its performance on the primary task. This occurs because the auxiliary task provides additional information that can be used to improve the model's understanding of the primary task, which can lead to better generalisation and faster learning [28].

2.2.3.1. Auxiliary Rewards

When privileged information is available, it can be used to construct auxiliary rewards that guide the agent toward desirable behaviors by evaluating performance against sub-goals. For example, in autonomous racing, privileged knowledge of track layout or opponent positions can generate bonuses for staying centered or avoiding near-misses.

While auxiliary rewards act as a form of reward shaping - supplying denser feedback to accelerate learning - there is a crucial distinction between auxiliary rewards and reward shaping. Traditional potential-based shaping, where the agent is, for example, rewarded for moving towards a target and penalised for moving away from it, guarantees that the underlying task and optimal policy remain unchanged [29]. Auxiliary rewards, however, act as heuristic bonuses that are rarely potential-based. Consequently, they risk altering the task being solved, which can cause the agent to converge on suboptimal policies for the original objective [29].

2.2.3.2. Auxiliary losses

The incorporation of *auxiliary heads* that predict privileged information into a model, and use of supervised *auxiliary losses* to train them, has emerged as a promising technique for improving the performance, interpretability, and training time of end-to-end learning approaches. It does so in three ways: it reduces the variance of the learning signal, it allows privileged information to be used even when it is unavailable at inference time, and it improves representation quality [8], [30].

Examples of auxiliary learning with privileged information prediction include Lample et al. [31] who demonstrated outstanding performance in first-person shooter games by training an agent to predict the presence of enemies in the agent's field of view (FOV). They further found that auxiliary losses performed badly on feedforward models, where classification accuracy was limited, but that they were effective on a recurrent model. Mirowski et al. [32] who found auxiliary tasks for depth prediction and loop closure detection improved navigation in 3D mazes, even compared to agents trained with depth information directly as input.

In the field of autonomous driving, Chitta et al. [33] applied auxiliary losses for depth prediction, semantic segmentation, and vehicle detection, achieving state-of-the-art performance on the CARLA benchmark, and decreasing the crash rate. Chen et al. [7] use auxiliary losses for birds-eye-view prediction in urban environments, improving both interpretability, and performance.

2.3. Evaluation of existing approaches

End-to-end RL agents on the RoboRacer platform generalise well to unseen tracks but have yet to match classical pipelines, and their opaque representations make failures hard to diagnose. For example, one may not understand whether an agent chose to accelerate into a wall because it misjudged its current velocity, or because it thought the track was straighter than it actually is. In simulation, privileged state such as velocity, tyre slip and upcoming track geometry is available but cannot be used at deployment, when these quantities are unavailable. Existing ways of exploiting privileged information during training each have a limitation: An asymmetric critic improves value estimation but doesn't directly shape the policy's own representation; teacher-student distillation is complex since it requires a separate teacher and a two-stage pipeline; and auxiliary rewards alter the optimisation objective and must still pass through the high-variance policy gradient.

Auxiliary losses avoid all three: they leave the RL objective unchanged, supply a lower-variance supervised gradient directly to the shared representation, and are simple to implement. They have improved performance in first-person games, 3D navigation and urban driving. However, to our knowledge, they have not been studied for LiDAR-only racing, nor has it been examined which privileged targets help, or how they change what the policy's representation encodes. This project addresses that gap.

Chapter 3

System overview

The system trains an end-to-end RL agent to drive a simulated RoboRacer vehicle from as input LiDAR. It runs entirely in simulation and has two subsystems, shown in Figure 3.1: a simulator, which models the vehicle, track and physics interactions, and a Python training script, which computes rewards given the state communicated by the simulator, and updates the policy. The two communicate over gRPC through the ML-Agents toolkit [34], [35].

At each timestep t, the simulator sends the current environment state s_t to the training script. The training script then computes the observation o_t using the state information s_t and performs a forward pass through the policy using o_t as input, computing the next action a_t. This action is sent to the simulator, and is applied to the vehicle, stepping the environment forward to the next timestep $t + 1$.

To speed up training, multiple instances of the simulator are run in parallel, each with its own set of agents running in separate sandboxed simulation environments within the same simulator process.

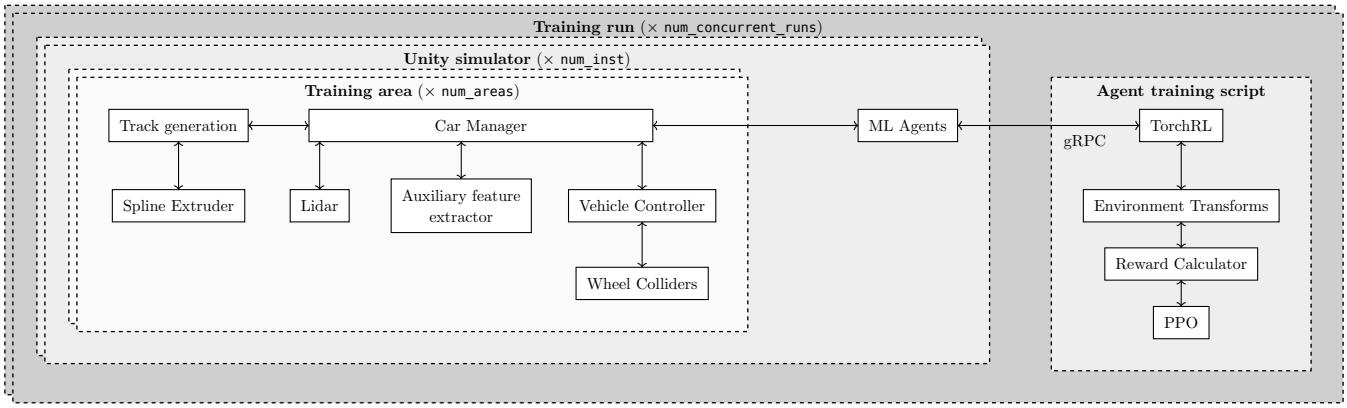


Figure 3.1: System diagram. There are `num_concurrent_runs` concurrent training runs running simultaneously, each with `num_inst` parallel simulator instances. Each simulator instance simulates `num_areas` vehicles, each with its own track. The simulator communicates with the Python training script over gRPC, and the training script updates the policy using PPO.

The following chapters shall outline the design and implementation of the two subsystems: the simulator (Chapter 4) and the training script & agent (Chapter 5).

Chapter 4

Simulator

Previous work on end-to-end driving has largely made use of the F1Tenth gym, with the F1Tenth gym being the most common simulator among methods surveyed in the survey by Evans et al. [11]. The F1Tenth gym is a Python-based 2D simulator for the F1Tenth (now RoboRacer) vehicle. This simulator is easy to use, and runs in Numba-compiled Python, but features a simplified bicycle model of the vehicle, which approximates slip dynamics using a linear function [10].

The AutoDrive simulator [36], which uses the Unity game engine [37], is a 3D simulator that models the vehicle and its environment using PhysX [38], a physics engine developed by NVIDIA. PhysX simulates rigid-body dynamics, including collisions, friction and gravity, and allows for higher-fidelity simulations, including weight transfer dynamics, and suspension deformation in Unity.

Table 4.1: Feature comparison of the Unity (PhysX) and F1Tenth gym simulators

	Unity (PhysX)	**F1Tenth gym**
Integrator	Euler	Runge-Kutta 4th order
Car model	3D, 4-wheel rigid body with suspension, weight transfer, non-linear wheel slip dynamics	2-wheel bicycle model with linearised slip dynamics
Sensors	LiDAR (max 101 rays per agent), IMU, wheel encoders[1], occupancy grids	LiDAR, IMU (excl. pitch, roll)
Track generation	No	In Beta [39]

Despite its higher-accuracy physics, the AutoDrive simulator by default does not provide various features that are important for end-to-end training in our auxiliary learning setup. For example, it is missing track generation & privileged information reporting. Therefore, a custom simulator was implemented in Unity using components from the AutoDrive simulator. Specifically, the `VehicleController` and `IMU` components were not re-implemented, but instead used directly from the AutoDrive simulator. The rest of this chapter describes the custom simulator, including its performance, vehicle model, track generation, and privileged information reporting.

[1]The IMU and wheel encoders are modules available in the AutoDrive simulator, not native Unity

4.1. Simulator performance comparison

In simulation-based training, throughput is critical because agents typically require a very large number of timesteps to learn. There are two kinds of timesteps: at each *control timestep*, the agent receives an observation and outputs an action, and at each *physics timestep*, the physics engine advances the numerical physics integration. The simulation timestep is usually shorter than the control timestep, since the dynamics must be integrated at a finer resolution to remain accurate. Following the settings used by Evans et al. [11], we use a control frequency of 25 Hz and a simulation frequency of 100 Hz.

In order to ensure the custom Unity simulator is suitable for training and not just evaluation, its performance was compared to the F1Tenth gym. A rollout of 10 000 control timesteps (6 minutes and 40 seconds in simulation time) was performed in each environment on an M4 MacBook Air, and the average time per control step was measured. Both environments were run in headless mode to remove any potential overhead from the graphical user interface.

Despite the Unity simulator being much more complex than the F1Tenth gym, it runs at a comparable speed, using a comparable amount of CPU resources in this environment, as shown in Figure 4.1, with the Unity simulator being only 19% slower compared to the F1Tenth Gym in terms of mean steps per second (sps).

Due to the relatively minor performance difference, and Unity's more realistic 3D physics, the Unity simulator was chosen for training the RL agent, despite the F1Tenth gym being more widely used in the literature.

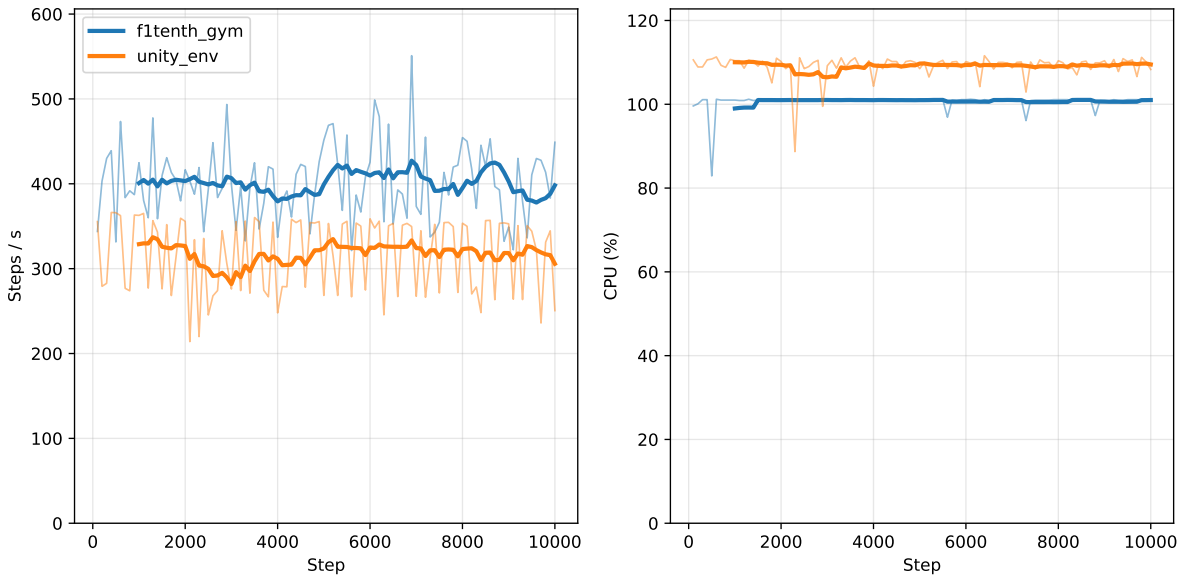


Figure 4.1: Benchmark of Unity simulator vs F1Tenth gym. In the CPU chart, a value greater than 100 indicates that the simulator used more than one CPU core. Includes child processes spawned by the simulator (e.g. in Unity's case)

4.2. Unity Vehicle Model

The vehicle is modelled as a `RigidBody` with four wheels, each simulated by a `WheelCollider`. The two components divide the work: the `Rigidbody` represents the car as a single moving mass, and the `WheelCollider`s generate the forces that move it. Near the front of the car, a `RayPerceptionSensor` from the ML-Agents toolkit simulates a 2D LiDAR sensor with 101 rays, which is used as the agent's observation.

4.2.1. The RigidBody

A `RigidBody` is the Unity component that gives an object rigid physical behaviour. It stores the car's mass, centre of mass and moment of inertia. At each fixed timestep the physics engine (PhysX) integrates the forces and torques acting on it to update its linear and angular velocity and its pose. The car's body is covered by simple box colliders (Figure 4.2), which handle collisions with walls and other objects. They play no part in the tyre-road interaction, which is handled entirely by the wheels. `BoxColliders` were used to model the car's body and collisions between the wheels and walls, since the `WheelCollider` component doesn't detect these kinds of collisions. The mass of the vehicle is set to 3.47 kg based on the values reported by Samak and Samak [40].

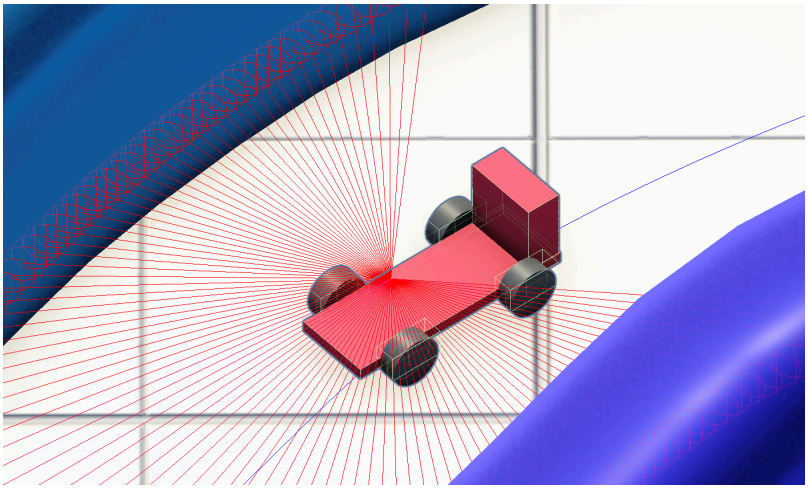


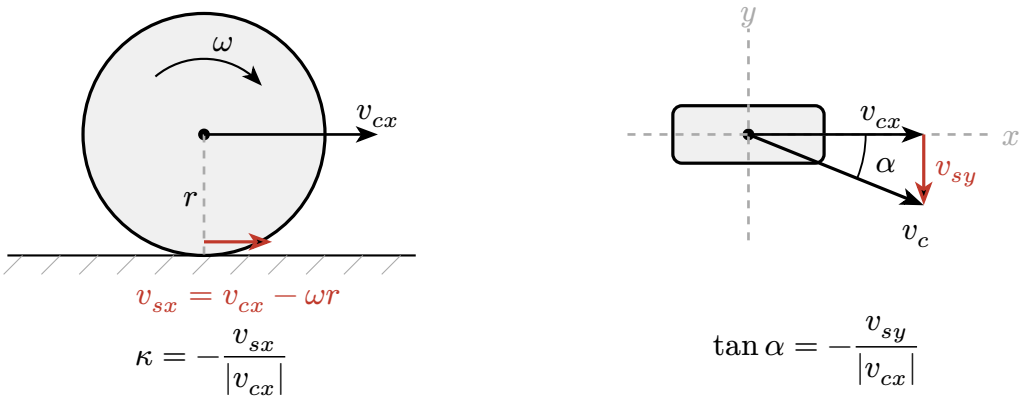
Figure 4.2: Custom Unity car's colliders: the body colliders handle collisions, and the four `WheelCollider`s handle suspension and tyre forces. The red lines show the raycasts by the `RayPerceptionSensor` that simulates the LiDAR. Despite the car looking simpler than the one in the AutoDrive simulator, it has similar physical properties, since we removed the visual meshes and textures.

4.2.2. The WheelCollider

In our custom simulator, each wheel of the RoboRacer is modelled as a `WheelCollider`, which simulates the wheel's suspension, rotation and tyre friction [41]. This kind of collider uses physics similar to that modelled by the semi-empirical "magic formula" wheel dynamics proposed by Pacejka et al. [42], [43].

The tyre model is built on the idea of *slip*. *Longitudinal slip*, κ, is the ratio between the longitudinal slip velocity and the wheel centre velocity. *Lateral slip*, $\tan(\alpha)$, is the

ratio between the lateral slip velocity and the wheel centre velocity. A wheel slips when it spins faster than the wheel centre moves (wheelspin), turns slower than the wheel centre moves (braking), or is dragged sideways (cornering or drifting). A slip value of 0 therefore indicates a freely rolling wheel, while a slip value of > 1 indicates that the point of contact has a higher velocity than the wheel centre.



(a) Side view: longitudinal slip **(b)** Top view: lateral slip

Figure 4.3: Slip quantities. Longitudinal slip compares the slip velocity at the contact point with the wheel-centre velocity; lateral slip compares the sideways slip velocity with the forward velocity.

A tyre needs some slip to produce force. The `WheelCollider` computes the force from the slip using a friction curve (Figure 4.4). Force rises with slip until it reaches a peak, the *extremum*, which is the tyre's maximum grip. Beyond the peak, the tyre begins to slide and the force falls to a lower, constant value, the *asymptote*. Pushing the car harder therefore gives less force, not more, which is a regime a racing agent must learn to avoid. The curve is defined separately for the forward direction (accelerating and braking) and the sideways direction (cornering). Note that in real life, at low slip values, the rubber stretches instead of actually sliding on the road. This is modelled by the rising part of the curve at low slip values.

Figure 4.4 shows the difference between a Unity-style piecewise friction curve and a Pacejka "magic formula" curve that was fit using least-squares to approximate Unity's friction curve. The values used for our simulations are detailed in Appendix D and are the values calibrated by Samak and Samak [40] for the RoboRacer platform, ported to our custom Unity simulator.

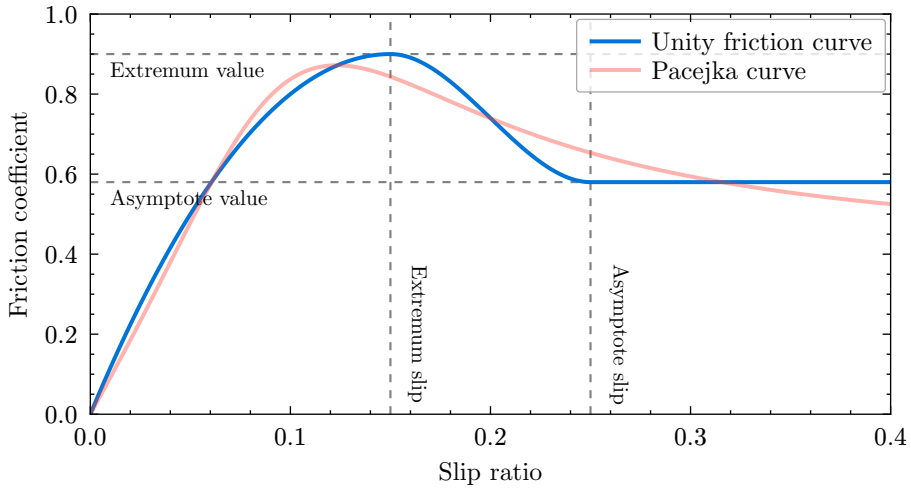


Figure 4.4: Approximated WheelCollider friction curves [41] for the RoboRacer compared to the Pacejka "magic formula" [42]

4.2.3. Steering

On the real-life RoboRacer, steering actuation cannot occur instantaneously, but is limited by the steering servo's maximum angular velocity. The `WheelCollider` component does not model this limitation, so we use the `VehicleController` from AutoDrive to implement a cap on steering speed of $\omega_{\max} = 315°/\,\text{sec}$. This value was also chosen based on the calibration of the RoboRacer platform by Samak and Samak [40].

4.2.4. LiDAR

The vehicle is equipped with a simulated 2D LiDAR sensor, which is implemented using a `RayPerceptionSensor` from the ML-Agents toolkit. The sensor emits 101 rays in a fan pattern, covering a field of view of 270°. Each ray measures the distance to the nearest obstacle in its cast direction, and returns a normalized value between 0 and 1, where 1 indicates no obstacle within the maximum range of the sensor (20m).

4.3. The track

During training, a variety of tracks are used to prevent the policy from overfitting to any single layout. Training on diverse environments has been shown to improve generalisation to unseen environments [44], and is closely related to domain randomisation, where simulator parameters are randomised to produce more robust policies [45], [46].

4.3.1. Centerline generation

To this end, we implemented a procedural track generator in C# in Unity. The idea is a *mean-reverting random walk*: a polyline of N equal-length segments whose heading wanders randomly but is continually pulled back towards the heading of a *reference circle* of radius R. The result wanders freely but still forms a single closed loop of roughly predictable size.

The walk keeps track of two angles: the *reference heading* φ_i, the tangent direction of the reference circle, and a *heading offset* δ_i relative to it. The segment direction is $\theta_i = \varphi_i + \delta_i$. Only the offset is random; the reference heading advances deterministically by $2\pi/N$ per step.

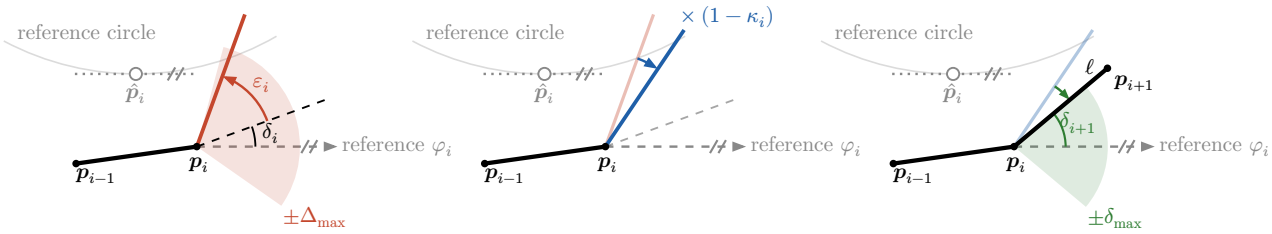
The walk starts at $p_0 = (R, 0)$ with $\delta_0 = 0$, i.e. tangent to the circle. Each segment has length $\ell = 2\pi R/N$, the circumference divided by the number of segments. For $i = 0, 1, ..., N-1$, with progress $t_i = (i+1)/N$:

1. We calculate the reference heading, $\varphi_i = 2\pi t_i + \pi/2$, the tangent of the reference circle at $\hat{p}_i = R \cdot (\cos 2\pi t_i, \sin 2\pi t_i)$.
2. We take a random turn (Figure 4.5 (a)), drawing $\varepsilon_i \sim \mathcal{U}(-\Delta_{\max}, \Delta_{\max})$ and adding it to the offset.
3. As depicted in Figure 4.5 (b), we shrink the offset towards zero by a factor $1 - \kappa_i$, where

$$\kappa_i = \kappa_0 + (1 - \kappa_0) t_i^s.$$

   The restoring strength κ_i starts at κ_0 and ramps up to exactly 1 on the final step. The exponent s controls how late the ramp kicks in.
4. In Figure 4.5 (c), we limit the offset from the reference heading to prevent accumulation of large offsets, and advance by length l:

$$\delta_{i+1} = \mathrm{clip}((1 - \kappa_i)(\delta_i + \varepsilon_i), -\delta_{\max}, \delta_{\max}),$$
$$p_{i+1} = p_i + \ell(\cos(\varphi_i + \delta_{i+1}), \sin(\varphi_i + \delta_{i+1})).$$



(a) Random turn (b) Mean reversion (c) Clip and step

Figure 4.5: One step of the track-generation walk. **(a)** A random turn $\varepsilon_i \sim \mathcal{U}(-\Delta_{\max}, \Delta_{\max})$ is added to the carried offset δ_i, measured from the reference heading φ_i. **(b)** A fraction κ_i of the accumulated offset is removed, pulling the heading towards the reference φ_i. **(c)** The offset is clipped to $\pm\delta_{\max}$, giving δ_{i+1}, and a step of length ℓ is taken to $\boldsymbol{p}_{i+1}$.

Early on, $\kappa_i \approx \kappa_0$ is small, so the offset behaves like a weakly damped random walk (an AR(1) process) and the track meanders. Near the end, $\kappa_i \to 1$ forces the heading back onto the reference tangent, so the walk heads towards the start point. Because the integrated heading still deviates slightly from the circle, p_N generally misses p_0 as visualised in Figure 4.6 (a). We close it by distributing the gap linearly along the track:

$$p_i \leftarrow p_i + \frac{i}{N}(p_0 - p_N), \quad i = 0, ..., N.$$

The gap is (usually) small, so this correction barely distorts the track. Finally, we fit a Catmull-Rom spline to the polyline to smooth out the corners, as visualised in Figure 4.6 (c).

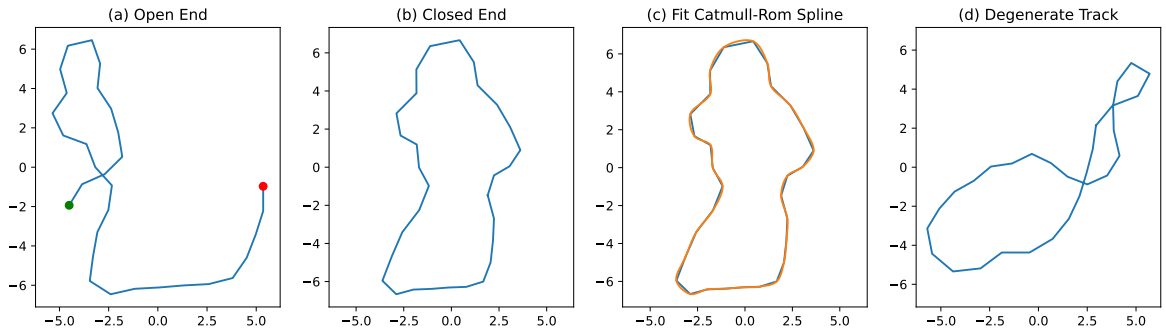


Figure 4.6: (a) Open end of the track before distributing the gap between p_0 (green) and p_N (red) **(b)** Closed track after distributing the gap **(c)** Catmull-Rom spline interpolation of the polyline **(d)** A degenerate track

This method is not immune to generating self-intersecting (degenerate) tracks, as shown in Figure 4.6 (d), but the probability is low enough that we simply discard any degenerate tracks and generate a new one. Empirically we find 50 tracks of 100 000 to be degenerate with our chosen parameters, so a 95%-confidence interval (C.I.) for $P(\text{degenerate track} \mid N = 30, \Delta_{\max} = 80°, \kappa_0 = 0.1, s = 3, \delta_{\max} = 90°)$ is $(0.037\%, 0.065\%)$ assuming a binomial distribution.

4.3.2. Track walls

Adding track walls to the generated centerline spline is a non-trivial problem. A naive approach is to scale the centerline's vertices outwards by a fixed multiple, or provide a fixed offset from the centre, but this can create undrivably narrow sections, as visualised in Figure 4.7 (a) and (b) respectively. We therefore generate the walls using the Clipper2 library [47], which implements a polygon offsetting algorithm inspired by Vatti [48]. The Clipper2 algorithm is robust to self-intersections, but may produce inset "islands" for large inset distances, or for tracks which are close to self-intersecting, as visualised in Figure 4.7 (c).

Empirically, we find that 1350 of the 100 000 tracks produce these islands (C.I. $(1.28\%, 1.42\%)$), so we simply discard any tracks with islands and re-generate them.

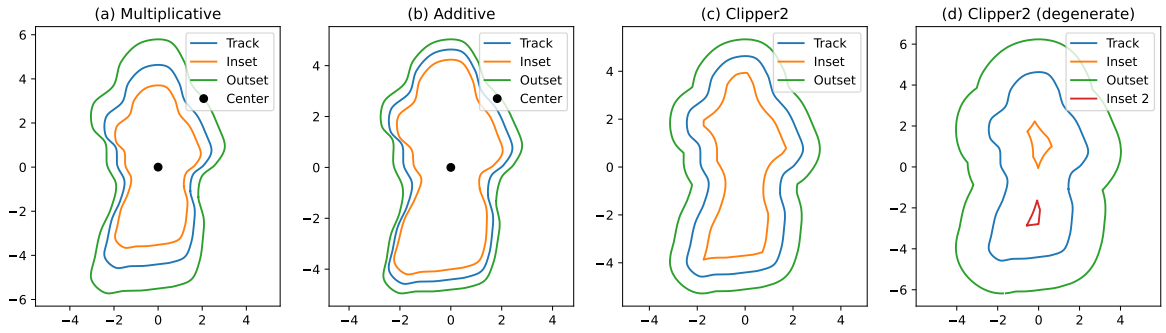


Figure 4.7: Track insets and outsets using different methods

4.3.3. Track Curvature

During training, we wish to observe the effect of using track curvature as an auxiliary loss. To this end, we compute the curvature of the track at 16 equally-spaced points along the track ($d \in \{0.5, ..., 8\}$ m) ahead of the car's current position. The curvature is computed using the formula $c_d = \frac{1}{r_d}$, where r_d is the radius of the osculating circle at that point on the track. The osculating circle was introduced by Leibniz [49] as cited by [50, p. 419, n. 6]; it is the circle that best approximates a curve at a given point, and its radius is computed from the first and second derivatives of the track's parametric equations [51, p. 910-911]:

$$\frac{1}{\kappa} = r = \frac{\left(x'(t)^2 + y'(t)^2\right)^{\frac{3}{2}}}{|x'(t)y''(t) - y'(t)x''(t)|} = \frac{|\boldsymbol{r}'(t)|^3}{|\boldsymbol{r}'(t) \times \boldsymbol{r}''(t)|}$$

Chapter 5

The Agent

In this chapter, we describe the agent architecture and the auxiliary losses used to improve its performance. To compare the effect of each auxiliary loss, we train two agent architectures and run a leave-one-in experiment on each. For each architecture, we train a baseline with no auxiliary losses, plus one run per group of auxiliary losses in which only that group is enabled, so each group's contribution can be measured against the baseline.

We shall first formalise the problem of autonomous racing as a partially observable Markov decision process (POMDP) in Section 5.1, then describe the process of reinforcement learning in Section 5.2, describe the architectures of the agent in Section 5.3, and discuss how the weights of the auxiliary losses are chosen in Section 5.4. Finally, Section 5.5 describes the metrics used to evaluate the performance of the agent.

5.1. Partially Observable Markov Decision Processes (POMDPs)

In order to formally describe the problem of autonomous racing, we first introduce the framework of partially observable Markov decision processes (POMDPs). POMDPs are a generalisation of Markov decision processes (MDPs) that account for the fact that an agent may not have access to the full state of the environment. In a POMDP, the agent receives observations that provide partial information about the underlying environment state, and must make decisions based on this incomplete information.

A POMDP is defined by the tuple $(\mathcal{S}, \mathcal{O}, \mathcal{A}, \mathcal{P}, \rho_0, \mathcal{R}, \gamma, \mathcal{Z})$, based on the work by Kaelbling et al. [52]:

At each time step t, the environment is in state $s_t \in \mathcal{S}$, where $\mathcal{S}$ is the state space. This state is a *complete* description of the environment at that time. The first state s_0 is sampled from an initial state distribution ρ_0. The agent does not see s_t directly. Instead, it receives an observation $o_t \in \mathcal{O}$, typically its sensor inputs (e.g. LiDAR, camera frames), drawn from the observation function, $o_t \sim \mathcal{Z}(\cdot \mid s_t)$: the observation function is a stochastic mapping from the true state to the observation, deciding what the agent observes given the true state. The information in s_t that is missing from o_t is called *privileged information*. For example, the car may observe its lidar measurements, but not the road curvature ahead directly, which is privileged information.

Based on its observations, the agent uses its policy π to choose an action $a_t \in \mathcal{A}$, where $\mathcal{A}$ is the action space. The policy therefore maps the history of observations[2] to a probability distribution over actions; we write[3] $a_t \sim \pi(\cdot \mid o_0, ..., o_t)$. The policy is what we want to find: it describes the agent's behaviour. After the agent has taken action a_t, the environment moves to a new state, $s_{t+1} \sim \mathcal{P}(\cdot \mid s_t, a_t)$, where $\mathcal{P}$ is the transition function. By the *Markov property*, the next state s_{t+1} depends only on the current state s_t and action a_t, not on earlier history [18]. Then, the agent receives a scalar reward $r_t = \mathcal{R}(s_t, a_t, s_{t+1})$, where $\mathcal{R}$ is the reward function, together with the next observation o_{t+1}. This interaction loop is illustrated in Figure 5.1.

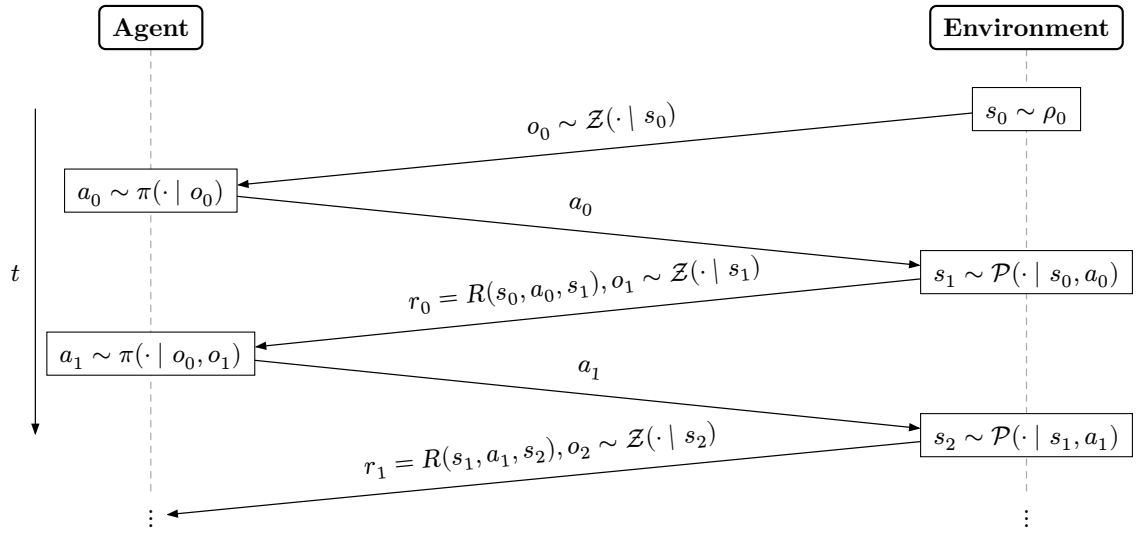


Figure 5.1: Interaction between the agent and the environment over time (t), which flows downwards.

The agent's goal is to maximise the *expected discounted return*. The *discounted return* (or simply *return*) from time step t is the discounted sum of all rewards the agent receives from time t onward,

$$G_t = \sum_{k=0}^{T-t} \gamma^k r_{t+k},$$

where T is the final time step and $\gamma \in [0, 1]$ is the discount factor. A reward received k steps ahead is scaled by γ^k, so later rewards are worth less than immediate ones: when γ is close to 1 the agent is far-sighted, and when γ is close to 0 it is short-sighted. Since G_t depends on random states, observations and actions, it is a random variable, and the agent maximises its expectation, $\mathbb{E}[G_0]$, from the start of the episode. The policy that achieves this maximum is the *optimal policy* π^*.

[2]Should the policy be defined using a single observation as input, it may perform arbitrarily worse than when using a full history of observations [52]. The observations are not markov, only the states are.

[3]Some formulations may define the policy as a mapping from the true state estimate to actions instead, but this is not an end-to-end approach, since there must then exist a state estimator, $e : \mathcal{O}^m \to \mathcal{S}$. In that case: $\hat{s}_t = e(o_0, ..., o_t), a_t \sim \pi(\cdot \mid \hat{s}_t)$

5.1.1. Value functions

Value functions measure how good it is for the agent to be in a state, or to take an action in that state. The state-value function,

$$V^\pi(s) = \mathbb{E}_\pi[G_t \mid S_t = s]$$

is the expected discounted return starting in state s and then following policy π thereafter, and is what the critic estimates in the PPO training algorithm (Section 5.2.3). The action-value function,

$$Q^\pi(s, a) = \mathbb{E}_\pi[G_t \mid S_t = s, A_t = a]$$

measures the expected discounted return starting in state s, taking action a, and then following policy π thereafter. Their difference is the advantage function,

$$A^\pi(s, a) = Q^\pi(s, a) - V^\pi(s), \tag{1}$$

which measures how much better action a is than the policy's average action in state s, following policy π.

5.2. Reinforcement Learning (RL)

In order to find the optimal policy, π^* that maximises the expected discounted return, $J(\theta) = \mathbb{E}_{\pi_\theta}[G_0]$, we use reinforcement learning (RL) as introduced by Sutton and Barto [18]. In RL, the agent learns to maximise its expected discounted return by interacting with the environment and receiving feedback in the form of rewards. The agent updates its policy based on the observed rewards and transitions, gradually improving its performance over time. We use RL instead of supervised learning due to lack of labelled data from expert demonstrations.

So-called model-free RL algorithms fall into three broad classes: *Value-based* methods learn the optimal action-value function $Q^*(s, a) = \max_\pi Q^\pi(s, a)$ and act greedily with respect to it, choosing the action with the highest value in the current state at each timestep: $\pi(s) = \arg\max_{a \in \mathcal{A}} Q^\pi(s, a)$. *Policy-based* methods, on the other hand, learn a parameterised policy π_θ directly by ascending the estimated gradient of $J(\theta)$ with respect to θ, and do not need the Markov property of the observations or an $\arg\max$ over actions. They can also represent stochastic policies, which may be necessary for optimal behaviour in some POMDPs [53] (although there always exists a deterministic optimal policy in fully-observed MDPs). *Actor-critic* [18] methods combine both: a learned value function (the critic) is used to reduce the variance of the policy gradient, and since the critic is only used in training, it may take privileged information as input. We focus on policy-based and actor-critic methods in this report.

5.2.1. Policy gradient theorem

Policy gradient methods are a class of RL training algorithms that learn a policy π_θ with parameters θ directly by maximising the expected return $\mathbb{E}[G_0]$ with respect to the policy parameters θ. Given a trajectory $\tau =$

$(s_0, o_0, a_0, r_0, s_1, o_1, a_1, r_1, ..., s_{T-1}, o_{T-1}, a_{T-1}, r_{T-1}) \sim p_\theta$, where p_θ is the probability of the trajectory when following the policy π_θ, we intend to maximise the expected return with respect to the policy parameters θ, so the objective function is

$$J(\theta) = \mathbb{E}_{\tau \sim p_\theta}[G_0] = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right] = \int p_\theta(\tau) G(\tau)\, \mathrm{d}\tau$$

Taking the gradient of $J(\theta)$ with respect to θ and applying the log-derivative trick [54], we obtain the policy gradient theorem[4] [55]:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid o_t) G_t\right]. \tag{2}$$

Equation 2 shows in what direction to update the policy parameters in order to increase the expected return. Of course, the expectation cannot be computed exactly without formulating the probabiltiy distribution over all possible trajectories, which is intractable, so we instead rely on estimates such as those detailed in Section 5.2.2 and Section 5.2.3.

5.2.2. REINFORCE

The REINFORCE algorithm for policy learning was introduced by Williams [54]. It is one of the simplest policy gradient methods: by sampling N trajectories, $\left\{\tau^{(1)}, ..., \tau^{(N)}\right\} \sim p_\theta$, of length T_i timesteps each, the policy gradient can be estimated in a Monte-Carlo fashion:

$$\widehat{\nabla_\theta J}(\theta) = \frac{1}{\sum_{i=1}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \nabla_\theta \log \pi_\theta\left(a_t^{(i)} \mid o_t^{(i)}\right) \cdot G_t^{(i)}$$

Although REINFORCE is simple to implement, it has several drawbacks, including having high variance in its gradient estimates, which can lead to slow convergence and instability during training [56].

5.2.3. Proximal Policy Optimisation (PPO)

PPO is a modern actor-critic algorithm for policy learning introduced in 2017 by researchers at OpenAI [57]. It improves on REINFORCE in two ways. First, being an actor-critic method, it learns a value function $V^\pi(s)$, which is used to compute an advantage estimate $\hat{A}_t$ and so reduces the variance of the policy gradient estimate, $\widehat{\nabla_\theta J}(\theta)$. Second, it optimises a *clipped surrogate objective*, $\mathcal{L}_{\text{PPO}}$, that limits how far each update can move the policy. This prevents large, destructive steps and makes several epochs of updates on the same rollout possible.

Recall that the advantage function in Equation 1 is the difference between the action-value and state-value functions. It therefore measures how much better (positive) or worse (negative) an action is than the policy's average action in that state. In training,

[4] The derivation of the policy gradient theorem is stated in Appendix E.

$\hat{A}_t = \hat{A}^\pi(s_t, a_t)$ is estimated from observed rewards and the learned value function, $\hat{V}^\pi(s)$, for example with Generalised Advantage Estimation [58]. In PPO, the clipped surrogate objective is given by

$$\mathcal{L}_{\text{actor}} = -\frac{1}{\sum_{i=1}^{N} T_i} \sum_{i=1}^{N} \sum_{t=0}^{T_i - 1} \min\Big(\rho_t(\theta) \cdot \hat{A}_t, \text{clip}(\rho_t(\theta), 1-\varepsilon, 1+\varepsilon) \cdot \hat{A}_t\Big)$$

where $\rho_t(\theta) = \frac{\pi_\theta(a_t \mid o_t)}{\pi_{\theta_{\text{old}}}(a_t \mid o_t)}$ is the ratio of the new policy's probability of taking action a_t given observation o_t to the old policy's, and ε is a hyperparameter that controls how much the policy is allowed to change in a single update. The clip function ensures that a policy update that would make $r_t(\theta)$ fall outside of the interval $[1-\varepsilon, 1+\varepsilon]$ is clipped to the nearest endpoint of that interval. This prevents large policy updates that could destabilise training. The min function, on the other hand, ensures that, while the policy's update is capped when an update makes a good action ($A_t > 0$) more likely, it is not protected from large updates if an update makes a bad action ($A_t < 0$) more likely. These two mechanisms together ensure that the policy is updated in a way that improves performance while avoiding large, destabilising changes that are common in other policy gradient methods [57].

Since PPO is an actor-critic method, it also learns a value function estimate, $\hat{V}^\pi(s)$, which is used to compute an advantage estimate $\hat{A}_t$ and so reduces the variance of the policy gradient estimate, $\widehat{\nabla_\theta J}(\theta)$. The value function is learned by minimising the mean squared error between the predicted value and the observed return. The full PPO loss function is therefore a combination of the clipped surrogate objective and the value function loss, along with an entropy bonus to encourage exploration. The final loss function is given by

$$\mathcal{L}_{\text{PPO}} = \mathcal{L}_{\text{actor}} + w_{\text{value}} \cdot \mathcal{L}_{\text{value}} - w_{\text{entropy}} \cdot \mathcal{H}[\pi_\theta]$$

where $\mathcal{L}_{\text{value}} = \left(\frac{1}{2}\right)\Big(\hat{V}^\pi(s_t) - R_t\Big)^2$ is the value-head loss function.

5.3. Neural Network Architecture

As discussed in Chapter 2, Lample et al. [31] found that auxiliary losses were more effective in a recurrent neural network (RNN), compared to a feedforward (FF) network in a first-person shooter game. Since it is not clear that this finding carries over to autonomous racing, we implement both a feedforward and a recurrent agent. Both model architectures are PPO actor-critic networks implemented in PyTorch [59], with the TorchRL library [60].

Each network consists of a shared *trunk* that produces a 64-dimensional feature vector, followed by three independent heads that all take this vector as input (Figure 5.2):

- the *policy head* outputs the parameters of the action distribution,
- the *value head* outputs the state-value estimate $\hat{V}^\pi(s)$, and
- the *auxiliary head* outputs predictions of the privileged targets

The trunk's input is a 50×101 tensor (or 4×101 in the RNN) containing the 50 (or 4 in the RNN) most recent LiDAR scans of 101 rays each. Four 1D convolutions slide along the ray axis and treat the time steps as input channels, so the first layer combines the full history at every angular position. The convolutions have 16, 32, 16 and 16 output channels, a kernel size of 8 rays (about 21° of the field of view) and ReLU activations. Their output is flattened, concatenated with the current steering angle and throttle, and passed through dense layers of 128, 72 and 64 units with ReLU activations.

The 64-dimensional output is then layer-normalised. Under heavy acceleration the car's body tilts, pointing the LiDAR above the track walls so that many rays return close to the maximum normalised length of 1. Without normalisation, these inputs can cause gradients to explode, which was observed in early tests.

In the recurrent variant, a gated recurrent unit (GRU) [61] with a hidden size of 64 follows the layer norm, and its output is passed to the heads instead. The GRU was used instead of a long short-term memory (LSTM) [62] because it simpler, and because previous work [63] has shown it to be more effective in predicting velocity state in autonomous racing, which is closely related to our task. The two network variants are otherwise identical, so any difference in performance can be attributed to the recurrence.

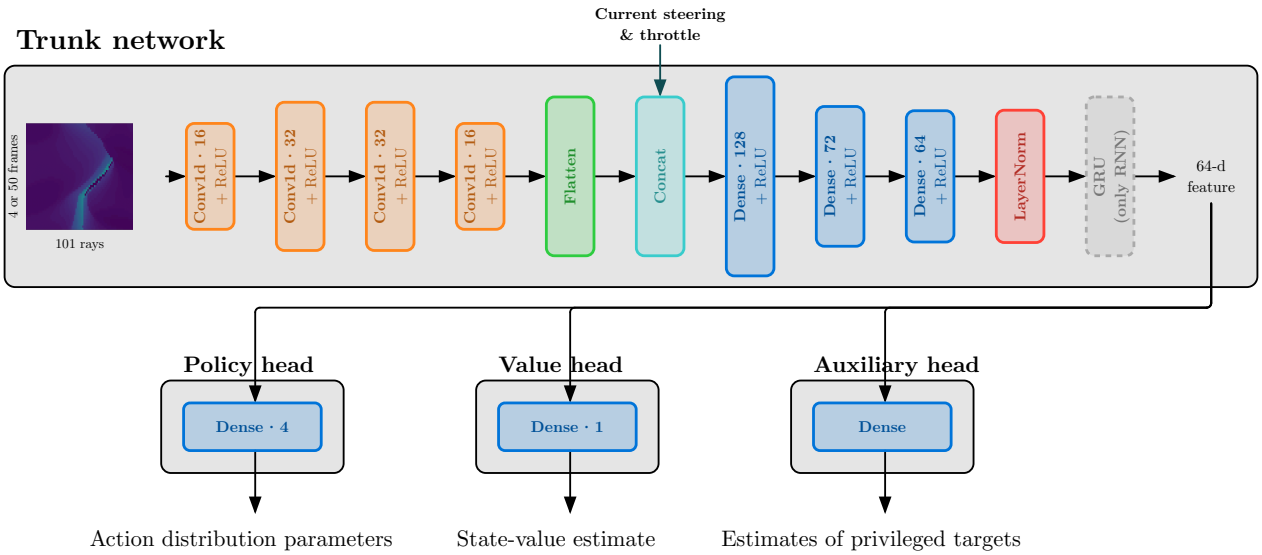


Figure 5.2: The agent's neural network. A LiDAR stack passes through four 1D convolutions; the flattened output is concatenated with the current steering and throttle and passed through three dense layers and a layer norm. After the GRU layer in the case of the RNN, a 64-dimensional feature vector is generated. This feature vector is then passed to three independent heads: the policy head outputs the parameters of the action distribution, the value head outputs the state-value estimate, and the auxiliary head outputs predictions of privileged targets.

5.3.1. Policy Head

The policy head outputs a location μ and a raw scale value x for each of throttle and steering. Each action is sampled independently from a tanh-squashed Gaussian,

$$a = \tanh(u), \quad u \sim \mathcal{N}\left(5 * \tanh\left(\frac{\mu}{5}\right), \sigma^2\right),$$

so that $a \in (-1, 1)$, where 1 corresponds to full throttle or full right steering, and -1 corresponds to full reverse or full left steering.

The mean is transformed with a tanh to soft-clamp it to the range $(-5, 5)$. This prevents the mean from growing too large, which would push samples into the saturated region of the squashing function and make it difficult for the policy to recover since gradients vanish there. The constant factor of 5 is chosen heuristically since it allows the model to use most of the action range ($\tanh(5) = 0.9999$), while being at the limit of a useful gradient ($\tanh'(5) = 1 - \tanh^2(5) \approx 1.8 \times 10^{-4}$).

The standard deviation is made positive with a softplus, with a small floor to ensure that the distribution is never degenerate: $\sigma = \text{softplus}(x) + 10^{-3}$. Further, policy head's linear layer uses a weight initialisation with gain 0.01 and zero bias, so every output is close to 0 at the start. Hence $\mu \approx 0$, and both actions are centred near the middle of their range.

This is important because the derivative of the squashing $\tanh(u)$ function goes to 0 as $|u|$ grows. If the head's outputs start large, samples land in the saturated region, the gradient of the policy loss with respect to μ is almost zero, and the policy cannot recover. In early tests this is exactly what happened: the car locked at full throttle and full steering in either the positive or negative direction.

5.3.2. Auxiliary heads

Using privileged information during training can improve the agent's performance, as discussed in Section 2.2. We incorporate it through *auxiliary losses*: additional loss terms that encourage the network to learn useful representations of the environment. These losses are computed from quantities that are available during training but withheld from the agent at test time. They therefore shape what the network learns without the agent relying on them at deployment. At every frame, the environment passes a set of privileged quantities to the trainer, which serve as auxiliary targets. After normalisation[5], one or more *auxiliary heads* learn to predict these targets, supervised by corresponding auxiliary losses. The auxiliary targets we consider are listed in Table 5.1. During training, the auxiliary losses are added to the PPO loss, and the gradients of the combined objective are backpropagated through the shared network:

$$\mathcal{L}_{\text{total}} = \underbrace{\mathcal{L}_{\text{actor}} + w_{\text{value}}\mathcal{L}_V - w_{\text{entropy}}H[\pi_\theta]}_{\mathcal{L}_{\text{PPO}}} + \sum_{k=1}^{K} w^{(k)}\mathcal{L}^{(k)}_{\text{aux}},$$

so that

[5]In order to ensure the targets are on a similar scale, the targets are normalised to $\sigma = 1, \mu = 0$ using a rollout generated from a pure PPO policy (i.e. without auxiliary losses/heads) as data to estimate σ and μ.

$$\nabla_\theta \mathcal{L}_{\text{total}} = \nabla_\theta \mathcal{L}_{\text{PPO}} + \sum_{k=1}^{K} w^{(k)} \nabla_\theta \mathcal{L}_{\text{aux}}^{(k)}$$

Table 5.1: The auxiliary targets to be predicted by the agent

Target	Units	Description
Forward velocity v_f	m/s	The component of the agent's velocity in the direction of its forward axis
Lateral velocity v_l	m/s	The component of the agent's velocity in the direction of its lateral axis
Yaw rate ω_y	rad/s	The rate of change of the agent's heading about its vertical axis
Track curvature ahead c_d	1/m	The curvature of the track centerline at d metres ahead of the agent, measured as the reciprocal of the radius of curvature
Per-wheel longitudinal slip κ_x	unitless	The longitudinal slip of wheel $x \in \{\text{FR}, \text{FL}, \text{BR}, \text{BL}\}$
Per-wheel lateral slip $\tan(\alpha_x)$	unitless	The lateral slip of wheel $x \in \{\text{FR}, \text{FL}, \text{BR}, \text{BL}\}$

5.3.3. Choice of Auxiliary Loss Function

The choice of auxiliary loss functions, $\mathcal{L}_{\text{aux}}^{(k)}$, has the potential to impact the agent's performance. Since we are predicting continuous quantities, it is natural to use a regression loss function such as Squared Error (SE), Absolute Error (AE), or Huber loss [64], defined respectively as

$$\mathcal{L}_{\text{SE}} = (\hat{y} - y)^2, \quad \mathcal{L}_{\text{AE}} = |\hat{y} - y|, \quad \mathcal{L}_{\text{Huber}}^{\beta}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 \text{ if } |y - \hat{y}| \leq \beta \\ \beta\ |y - \hat{y}| - \frac{1}{2}\beta^2 \text{ otherwise.} \end{cases}$$

Throughout, we use PyTorch's rescaled implementation of the Huber loss (`smooth_l1_loss`), $\frac{\mathcal{L}_{\text{Huber}}^{\beta}(y,\hat{y})}{\beta}$. Dividing by β decouples the threshold from the gradient: the rescaled loss is quadratic with gradient for small errors, but linear with a constant gradient of magnitude 1 for large errors.

The Huber loss is a robust alternative to the SE loss that is far less sensitive to outliers. It behaves quadratically for small errors and linearly for large ones, with the transition point set by a threshold parameter β. As visualised in Figure 5.3, this means the gradient magnitude is bounded no matter how large the prediction error grows, whereas the L2 gradient grows without limit. Unlike the L1 loss, whose gradient changes sign abruptly at zero, the Huber loss is smooth there, so the gradient shrinks gradually as the error approaches zero.

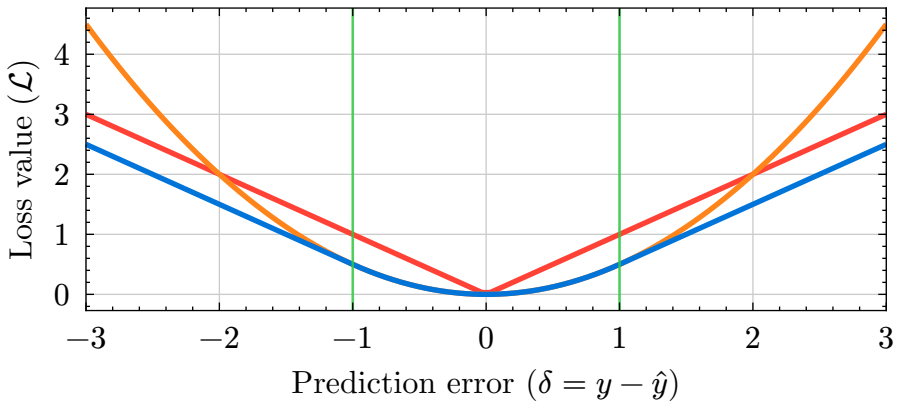


Figure 5.3: Rescaled Huber loss (blue) vs squared error loss (orange) vs absolute error loss (red) for prediction errors δ. Between the green lines, the Huber loss acts like the squared error loss, and outside, its gradient is equal to the gradient of the absolute error loss, so it is less sensitive to outliers than the squared error loss. The Huber loss is smooth at zero, unlike the absolute error loss, so its gradient shrinks gradually as the error approaches zero.

The Huber loss's bounded gradient is particularly useful when the target values have a heavy-tailed, non-normal distribution, since the loss is then far less sensitive to extreme values than the SE loss [65]. The majority of the auxiliary targets are heavy-tailed, with some having more than 27x the number of extreme extreme values ($|X| > 3\sigma$) compared to a normal distribution. A table with the statistics for each individual target is available in Appendix F.

For this reason, we use the Huber loss for all auxiliary targets. With $\beta = 1$, since the targets are normalised to zero mean and unit variance, this makes the loss quadratic for errors within one standard deviation of the mean - where most[6] of the fitting takes place - and linear, with a gradient of magnitude 1, beyond it, so that the extreme target values cannot dominate the gradient.

5.4. Finding the Optimal Weights

The question naturally arises: how should the weights $w^{(k)}$ of the auxiliary losses be set? If they are too small, the auxiliary losses will have little effect on the agent's learning. If they are too large, the agent may use all its capacity to optimise the auxiliary tasks and fail to learn a good policy, or the auxiliary task gradients may conflict with the main task gradients, slowing learning. This is an optimisation problem, where the weights are tuned to achieve the best overall performance. Two options exist: choosing the weights before training (hyperparameter tuning), or adapting the weights during training (adaptive weighting).

[6]Proportions of the data within 1 standard deviation of the mean are listed in Appendix F

5.4.1. Hyperparameter Tuning

Hyperparameter tuning is a common approach to finding the optimal weights for auxiliary losses. It involves running multiple training experiments with different weight configurations and then evaluating the agent's performance, choosing the configuration that yields the best results. This approach is simple to implement, but it can be computationally expensive and time-consuming. It also lacks a guarantee of finding the optimal weights, as the search space can be large and complex.

The simplest tuning method is a grid search, where a discrete set of weight values is defined for each auxiliary loss and all possible combinations are evaluated. However, this approach can be inefficient, especially when the number of auxiliary losses is large, as the number of combinations grows exponentially with the number of weights considered.

Random search [66] is another hyperparameter tuning method, where weight configurations are sampled randomly from a predefined distribution. This approach can be more efficient than grid search, as it allows for exploration of a larger search space without evaluating every possible combination. However, it still requires a significant number of training runs to find the optimal weights.

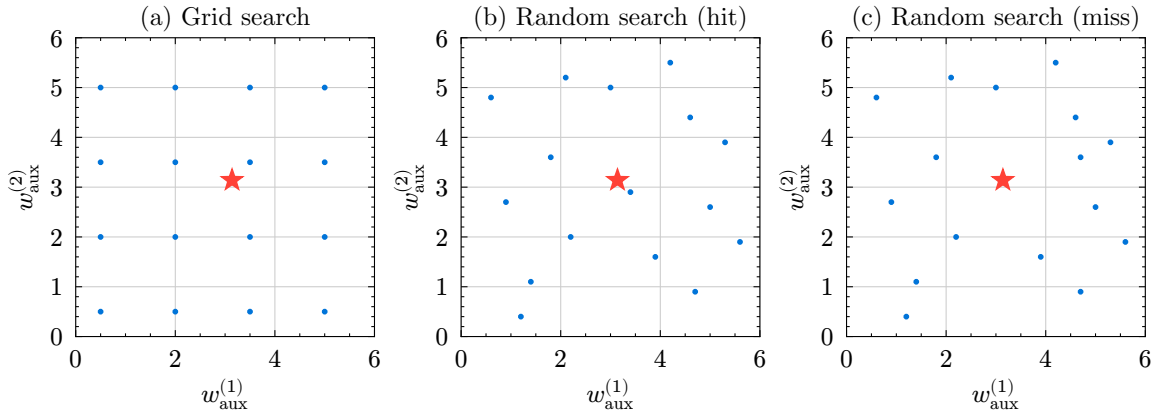


Figure 5.4: Grid and random search over two auxiliary loss weights. The red star indicates the optimal weight configuration. (a) shows that a grid search may miss the optimum, but is guaranteed to find a point within $\frac{\text{grid resolution}}{2}$ of the optimum. (b) shows that random search can find a point very close to the optimum, but (c) shows that it can also miss it entirely.

So-called "smart" hyperparameter tuning methods, such as Bayesian optimisation [67] and the tree-structured Parzen estimator (TPE) [68], build a model of the relationship between the weights and the agent's performance. This model is then used to guide the search for optimal weights, allowing for more efficient exploration of the search space. These methods can be more effective than grid or random search, but they can also be more complex to implement and still require multiple training runs. Methods such as Hyperband [69] and its asynchronous variant ASHA [70] mitigate this cost by giving each trial a small budget and only continuing the best-performing ones. However, such early-stopping methods may prematurely terminate RL trials that perform more exploration early on, despite performing better in the long run [71], so they are not ideal for our scenario.

5.4.2. Adaptive Weighting

Adaptive weighting is an alternative approach that dynamically adjusts the weights of the auxiliary losses during training based on factors like the magnitude of the gradients or the error of the auxiliary predictions. Various methods have been introduced in the literature, such as GradNorm [72], which weights tasks that are learning slower more heavily, and Uncertainty Weighting [73], which uses the model's uncertainty in each task to determine the weights. These methods are useful in multi-task learning scenarios, where the performance of all tasks is important, but they are not ideal for our scenario, where the main task (driving) is the only one that matters. In this case, we want to combine auxiliary losses in such a way that improves the main task's performance, not to balance the performance of all tasks.

5.4.3. Weighting Scheme Used in This Project

Our aim is to understand the effect of each kind of privileged information on the agent, not to find the best possible weights. Searching over a separate weight for each of the 16 curvature, 3 velocity and 8 slip targets would be intractable (Section 5.4), and adaptive methods are designed to balance tasks rather than to help a single primary task. We therefore use fixed weights and a *leave-one-in* design.

The targets are partitioned into $C = 10$ categories (Table 5.2), each with one scalar weight w_c. The auxiliary loss of a category is the sum of the Huber losses of its targets,

$$\mathcal{L}_c = \sum_{k \in c} \mathcal{L}^{(k)}_{\text{Huber}}, \quad \mathcal{L}_{\text{total}} = \mathcal{L}_{\text{PPO}} + \sum_{c=1}^{C} w_c \mathcal{L}_c.$$

In a leave-one-in experiment, a single category c^* is enabled with $w_{c^*} \in \{0.1, 0.3, 1.0\}$ and every other weight is zero, giving 30 configurations, each compared against the baseline ($w_c = 0$ for all c). The "All" configuration enables every category at the same w. All weights are constant throughout training, and since the targets are normalised to zero mean and unit variance, the per-target losses are of comparable magnitude. The limitation is that this approach does not explore interactions *between* categories.

Table 5.2: The auxiliary targets are grouped into categories, each of which is weighted by a single hyperparameter. The performance of the agent is evaluated for configurations with

Category	**Auxiliary targets**
None	No targets (i.e. baseline PPO)
Forward velocity v_f	Forward velocity v_f
Lateral velocity v_l	Lateral velocity v_l
Yaw rate ω_y	Yaw rate ω_y
Close-ahead track curvature	$\{c_x \mid x \in \{0.5, 1.0, 1.5, 2.0, 2.5, 3.0\}$ m$\}$
Medium-ahead track curvature	$\{c_x \mid x \in \{3.5, 4.0, 4.5, 5.0, 5.5, 6.0\}$ m$\}$
Far-ahead track curvature	$\{c_x \mid x \in \{6.5, 7.0, 7.5, 8.0\}$ m$\}$
Front wheel forward slip	$\{\kappa_x \mid x \in \{\mathrm{FR}, \mathrm{FL}\}\}$
Rear wheel forward slip	$\{\kappa_x \mid x \in \{\mathrm{BR}, \mathrm{BL}\}\}$
Front wheel lateral slip	$\{\tan(\alpha_x) \mid x \in \{\mathrm{FR}, \mathrm{FL}\}\}$
Rear wheel lateral slip	$\{\tan(\alpha_x) \mid x \in \{\mathrm{BR}, \mathrm{BL}\}\}$
All	All auxiliary targets

5.5. Metrics

In order to evaluate the agent's performance, we define a set of metrics that capture different aspects of its driving behaviour. These metrics are designed to provide insight into the agent's ability to navigate the track both (a) quickly and (b) safely.

5.5.1. Progress

Let L be the total length of the track centreline. At any time t, let $d_t \in [0, L)$ be the agent's current arc length from the start of the centerline to the point closest to the agent, and let N_t be the total number of completed laps. We define the agent's *absolute progress* along the track as the total continuous distance travelled from the start:

$$\xi_t = d_t + N_t L$$

The *progress per timestep* (PPT) is then simply the change in this absolute progress between consecutive timesteps, defined as $\xi_t - \xi_{t-1}$. Because ξ_t tracks the cumulative distance across multiple laps, this raw difference naturally handles crossing the finish line without requiring any wrapping operations or modulo arithmetic.

We also define the *progress per second* (PPS) as the progress per step divided by the control timestep duration (40 ms), which is a more intuitive measure of speed.

5.5.2. Collision Rates

The *collision rate per episode* is the fraction of total episodes that terminate in a collision rather than reaching the maximum length (45 simulated seconds).

The *collision rate per timestep* is the total number of collisions divided by the total number of timesteps executed across all episodes. This can be interpreted as an estimate of the probability of a collision occurring at any given timestep, and is useful for evaluating the agent's safety in a time-normalised manner.

The *collisions per metre of progress* (CPMP) is the total number of collisions divided by the total distance the agent successfully advances along the track centerline. This metric normalizes failures against progress rather than time, providing a balanced measure of the agent's driving competence.

5.5.3. Reward function

The agent receives a dense reward at each timestep that encourages fast, collision-free progress along the centerline:

$$r_t = w_p \cdot (\xi_t - \xi_{t-1}) - w_c \cdot c_t$$

The first term encourages forward motion, using the per-step progress measure defined in Section 5.5.1. The second term discourages collisions through the indicator c_t, which is 1 if the agent collides with an obstacle at time t and 0 otherwise. The weights w_p (reward per metre of progress) and w_c (penalty per collision) set the trade-off between speed and safety, and can be tuned to suit the application's risk tolerance. We use $w_p = 1.0$ and $w_c = 1.0$. A relatively small collision penalty is sufficient here because episodes end immediately on collision: the agent loses all future reward, which already acts as a strong implicit penalty for crashing.

5.5.4. Area Under the Curve (AUC)

The *area under the curve* (AUC) evaluates sample efficiency and learning stability by calculating the definite integral of a performance metric $y(t)$ (e.g., return or progress per step) over the total training time or timesteps T:

$$\text{AUC} = \int_0^T y(t)\,\mathrm{d}t$$

In practice, evaluation metrics are recorded at discrete time intervals $t_0, t_1, ..., t_N$, so the AUC is computed using the trapezoidal rule:

$$\text{AUC} \approx \sum_{i=1}^{N} \frac{y(t_{i-1}) + y(t_i)}{2}(t_i - t_{i-1})$$

A higher AUC for a measure like reward or progress indicates faster learning and superior performance throughout the entire training process rather than merely achieving high final performance.

5.5.5. R^2 score

The *coefficient of determination*, R^2, measures how much of the variation in a quantity a model can explain, compared to simply guessing the average value every

time. It is used in Section 6.2 to judge how well privileged quantities such as velocity or curvature can be predicted from the network's internal representations.

A score of 1 means the predictions are perfect, a score of 0 means the model does no better than predicting the average, and a negative score means it does worse. We report the *out-of-sample* R^2, computed on data the model was not fitted to, so it reflects how well the model generalises rather than how well it memorised its training data.

R^2 is used instead of error measures such as RMSE because it is unit-free. The quantities we predict have different units and scales (m/s, rad/s, 1/m and unitless slip), so their raw errors cannot be compared, whereas R^2 puts all of them on one scale.

5.6. Trainer Performance Tuning

Training was performed on a single machine with an 64 cores and 128 GB of RAM. To ensure experiments finish in a practical time, we first measured how best to use this hardware, aiming to maximise *aggregate* throughput: environment timesteps collected and learned from per second, summed across all models training simultaneously.

We ran a grid search over the parameters in Table 5.3: concurrent trainers (2, 4, 8, 12), PyTorch threads per trainer (8, 16) and parallel environments per trainer (4, 8, 10, 16). For each configuration, `concurrent_trainers` identical processes were launched at once, each running five PPO iterations (a 256-frame rollout plus four epochs of minibatch updates each iteration) with checkpointing and TensorBoard logging disabled. Configurations with more than 8 trainers and 16 environments (128 or more concurrent Unity instances) exhausted memory and were excluded, leaving 28 single-run configurations. Throughput per trainer was computed from the mean wall-clock time per iteration:

$$\text{throughput} = \frac{\text{rollout length} \times \text{parallel envs} \times \text{areas per env}}{\text{seconds per iteration}}$$

We report the throughput per trainer, showing how much each model slows as the machine gets busier, and summed across trainers, showing where adding trainers stops paying off.

Total throughput peaks at 8 trainers (4172 timesteps/s with 8 environments and 16 threads) and falls at 12 (at most 3853 timesteps/s), while per-trainer throughput roughly halves from 2 to 8 trainers. The best settings use $8 \times 16 = 128$ PyTorch threads in total, matching the hardware thread count; at 12 trainers the 16-thread configurations are oversubscribed and fall behind the 8-thread ones. We therefore chose (`concurrent_trainers`, `num_inst`, `pytorch_threads`) = (8, 8, 16), though its margin over the next-best configurations (4020 and 3962 timesteps/s) is within likely run-to-run variation. Using `num_areas` $\in \{8, 12\}$ per environment in this configuration did not help (3844 and 3999 timesteps/s). The performance of each configuration is available in Appendix I.

Table 5.3: Parameters varied in the training performance grid search.

Parameter	Values considered	Description
`concurrent_trainers`	$\{2, 4, 8, 12\}$	Number of independent models training simultaneously, each in its own process.
`pytorch_threads`	$\{8, 16\}$	Threads available to each trainer's PyTorch thread pools.
`num_inst`	$\{4, 8, 10, 16\}$	Number of Unity environment instances per trainer. Each contains 6 training areas, so each instance steps 6 agents in parallel.

Chapter 6

Experiments & results

To evaluate how auxiliary losses affect agent performance, we trained every combination of active auxiliary losses, auxiliary loss weight and model architecture for 600 PPO iterations. Each iteration collected 256 frames from each of 48 concurrent environments, so each run totals about 7.4 million timesteps ($\approx$ 82 hours of simulated time). We chose 600 iterations because a test run showed the baseline feedforward (FF) model's reward plateauing well before this point (Appendix H). Each configuration was trained with 6 different seeds.

To evaluate a run, we took the checkpoints from its last 50 iterations and ran each one for 614400 timesteps (~7 hours) on held-out, randomly generated test tracks. We then averaged each statistic over these 50 checkpoints to get one value per run. Averaging reduces checkpoint-to-checkpoint noise. Because consecutive checkpoints are strongly correlated, we treat this average as a single observation, since doing otherwise would lead to an overestimation of the sample size. Therefore, each configuration has $N = 6$ samples (one per seed) rather than 6×50.

6.1. Baseline Model Performance

The baseline feedforward model trained with the PPO loss and no auxiliary targets performed well in the primary task across $N = 6$ seeds. It achieved a mean CPMP across 6 seeds of 0.0034 ± 0.0028 crashes/m, indicating approximately one crash every 297 metres. It achieved a mean PPS of 4.13 ± 0.49 m/s $\approx$ 15 km/h.

The recurrent model performed significantly worse, with a mean CPMP of 0.0381 ± 0.0499 crashes/m, almost 10x higher than the feedforward model. It achieved a mean PPS of 2.00 ± 0.692739 m/s $\approx$ 7.2 km/h, approximately half the performance of the feedforward model.

Before we consider the effect of auxiliary losses, we first establish what the baseline models' internal representations encode in Section 6.2.

6.2. Learned Features of the Baseline Models

Consider the network in Figure 5.2. Once trained, each layer l maps an input $\boldsymbol{x}$ to an intermediate representation $\boldsymbol{h}^{(l)}$, which becomes the input to the following layer. We would like to know what information these intermediate representations encode, and in

particular whether they encode privileged state variables (e.g. velocity, slip, upcoming curvature) so we can better understand the model.

Alain and Bengio [74] introduced *linear probes* for this purpose; the trunk network's weights are frozen, and a separate linear classifier (the probe) is trained in a supervised manner on the representations, $\boldsymbol{h}^{(l)}$, of each layer to predict the desired target. Since the probe cannot change the representation, its test accuracy measures how much class information is *linearly* available at that layer. Anand et al. [75] extended this idea to reinforcement learning, probing an encoder's representations for ground-truth state variables of Atari games.

The probe deliberately linear; a sufficiently expressive probe could compute the target even from the raw LiDAR, so its accuracy would say more about the probe than about the representation [76]. Since our targets are continuous, we use linear *regression* probes, as McGrath et al. [77] and Gurnee and Tegmark [78] do.

A linear probe has the same form as our auxiliary heads: a single dense layer on the same 64-dimensional trunk output. The difference is that the probe is fitted by ordinary least squares (OLS) to a frozen trunk, while an auxiliary head is trained jointly with the policy, and its gradients reshape the trunk. A baseline probe's R^2 therefore approximates what an auxiliary head could achieve without changing the representation.

To establish what the baseline represents without auxiliary supervision, linear probes were fit to the output of the final trunk layer to predict each auxiliary target at various points during the model's training. Each probe is an OLS regression model trained on intermediate representations from a ~35-minute rollout (52429 timesteps) and evaluated on a separate ~9-minute rollout (13107 timesteps). Figure 6.1 reports out-of-sample R^2 at initialisation and after 600 PPO iterations.

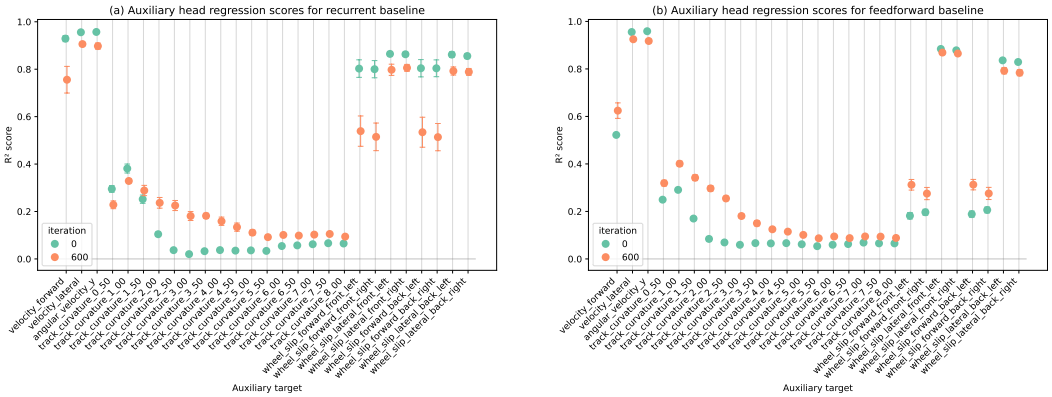


Figure 6.1: Out-of-sample R^2 scores of linear regression probing of the final layer of the trunk on the (a) recurrent baseline model and (b) feedforward baseline model. Error bars show the standard error of the mean (SEM) across $N = 6$ different seeds.

Three distinct patterns emerge, with notable differences between the feedforward (FF) and recurrent (RNN) baselines. Full values are reported in Appendix G.

First, quantities describing the vehicle's *instantaneous dynamics* are already linearly decidable at initialisation in both models. Lateral and angular velocity reach $\overline{R}^2 \approx$

0.955-0.959 in both. Forward velocity, however, is decodable far better from the RNN (0.929) than from the FF model (0.522). Since both trunks are untrained and He-initialised [79], this reflects information that is close to linear in the observation and survives random projections, rather than a learned representation. Alain and Bengio [74] observed a similar effect in intermediate layers of MNIST models[7]. Training does not improve the decodability of any of these quantities, except the forward velocity in the FF model. In the RNN all three fall, to 0.756, 0.906 and 0.897; forward velocity R^2 also varies considerably across seeds. We hypothesise that the agent can perform well without explicitly representing these quantities, so trunk capacity is better spent elsewhere, but that for the FF model, which has no memory, forward velocity is more important to represent explicitly.

Second, track curvature becomes more decodable with training at most horizons in both models. At initialisation, only close-ahead curvature is decodable. R^2 values are below 0.11 beyond 1.5m in both models. After training, the FF model improves at every horizon, reaching R^2 up to 0.30-0.40 up to 2.0m. The RNN instead loses some close-ahead decodability, but gains from 1.5m onward. It ends slightly ahead of the FF model at 3.5m-5.5m, although absolute values remain below 0.2 for both.

Third, wheel slip behaves differently in its forward and lateral components. Forward slip shows the largest difference between the models. It is only weakly decodable from the untrained FF model, but highly decodable from the untrained RNN. Training moves the two models towards each other: FF rises to 0.28–0.31, while the RNN falls to 0.51 –0.54. Lateral slip is highly decodable at initialisation in both models (0.83–0.88) and decreases with training. The decrease is consistently larger for the rear wheels of the FF model (0.79/0.78) than for the front wheels (0.87/0.86).

Overall, training of both models trades away linearly decodable instantaneous dynamics and slip in favour of more decodable track curvature beyond the immediate vicinity of the vehicle. The RNN starts with a richer representation of time-dependent quantities, but loses much of it during training. We hypothesise that forcing earlier learning of these quantities through auxiliary losses may improve the agent's performance, and we investigate this in Section 6.3 and Section 6.4.

6.3. Agent Performance After Auxiliary Learning

The models are now trained with the same reward function as the baseline model (Section 5.5.3), but with the addition of one or more auxiliary heads and their associated auxiliary losses, as described in Section 5.4.3.

Comparing the top-performing configurations of each architecture vs their respective baselines, we find that the feedforward model with the close-ahead curvature auxiliary losses at weight 1.0 achieved much more stable training performance than the baseline model, as qualitatively visualised in Figure 6.2 (a). The model also consistently outperformed its baseline in both PPS (Figure 6.2 (b)) and CPMP (Figure 6.2 (c)). The

[7]Alain and Bengio do not specify which initialisation scheme they use

recurrent model had much more variable training performance, but the close-ahead curvature auxiliary loss at weight 0.3 also consistently outperformed its baseline, and brought the recurrent model's performance closer to the level of the feedforward model.

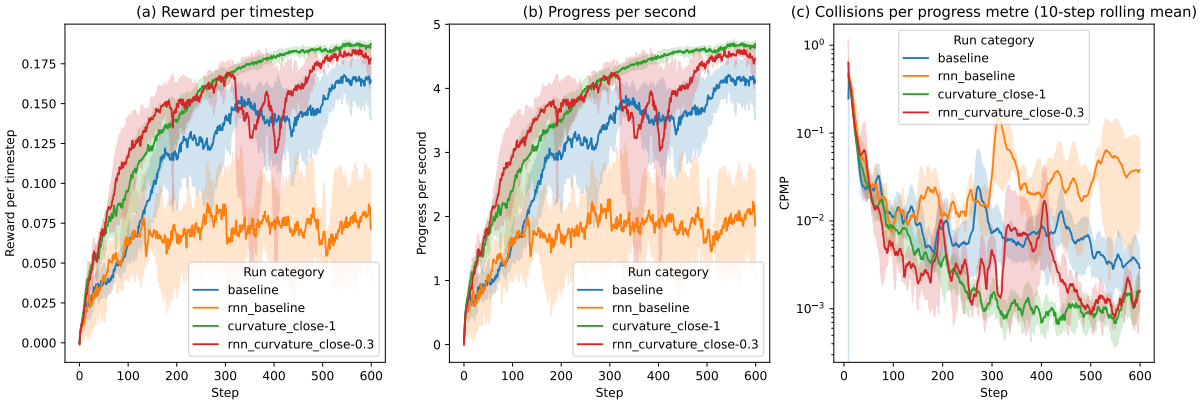


Figure 6.2: Training performance comparison of the baseline model and the best-performing auxiliary loss configurations for each architecture. The shaded area indicates the 95% confidence interval of the mean using bootstrapping [80]

Figure 6.3 compares the reward AUC of models trained with different auxiliary loss weights against the two baseline models. Some auxiliary losses improve the agent's training speed, while others degrade it. Notably, close-ahead curvature losses improve the AUC for both the feedforward and recurrent models, especially at higher weights, and these models also show some of the lowest variability across seeds. The baseline RNN model performed poorly, with a mean AUC well below the feedforward baseline. However, adding losses for forward wheel slip (at a weight of 0.1), close-ahead curvature (especially at weights 0.3 and 1), and all heads combined (with weight 0.1) improved it substantially. With the close-ahead curvature loss at weights of 0.3 and 1, the recurrent models exceed the feedforward baseline and nearly match the best feedforward model, closing the gap between the two architectures.

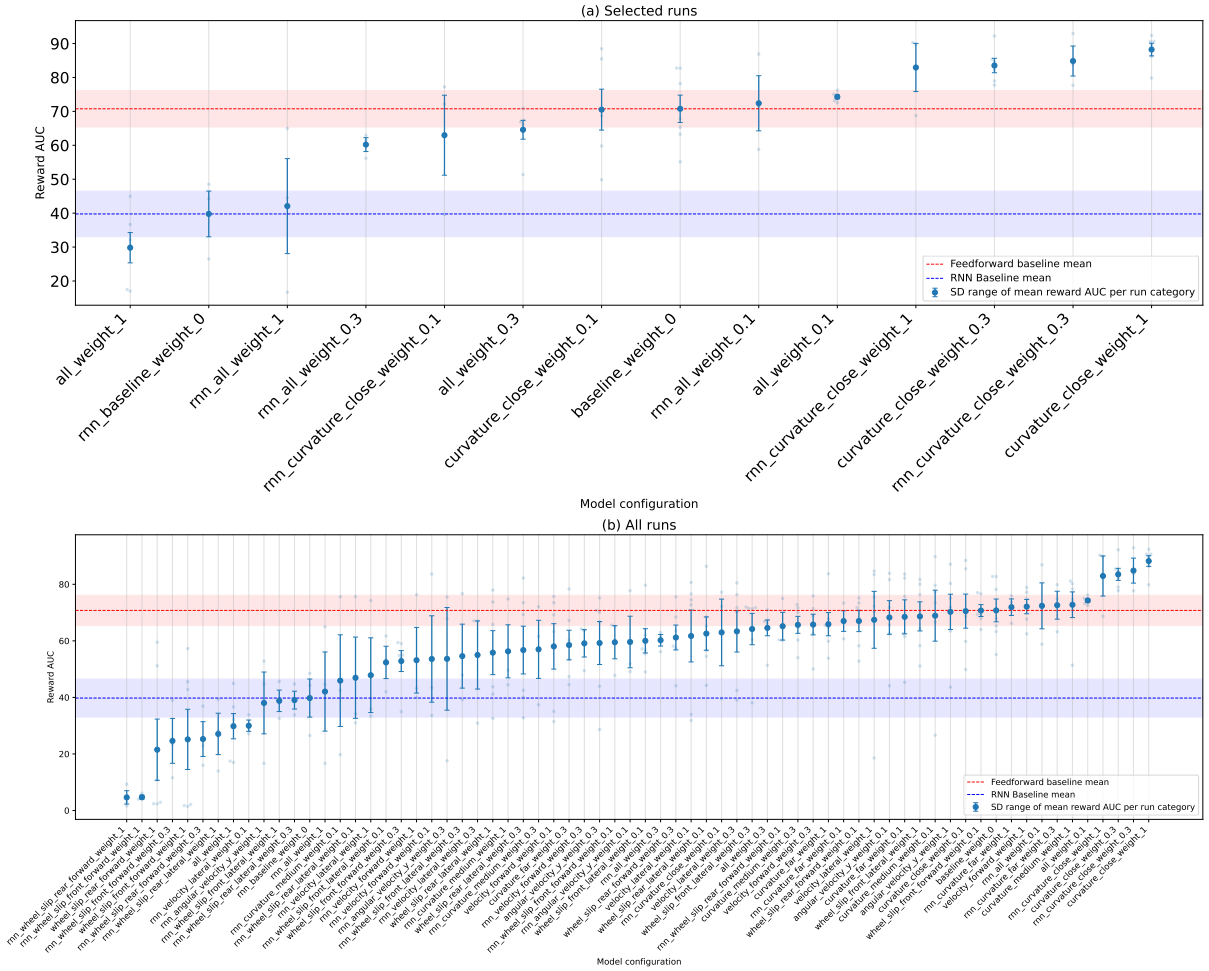


Figure 6.3: Reward AUC of the models trained with different auxiliary losses and loss weights over their 600 training iterations, compared to the baseline model (red). The standard deviation bars indicate the standard error of the mean across 6 different seeds each. (a) Shows only selected runs for clarity, while (b) shows all runs.

Figure 6.4 effect of the loss weight strongly depends on the auxiliary target. For close-ahead curvature, higher weights perform better: the feedforward close-ahead model goes from a decline of -11% reward per timestep at $w = 0.1$ to a significant reward improvement of $+13\%$, and a safety improvement of 62% at $w = 1.0$. On the other hand, at a weight of 0.1, the recurrent front and rear forward wheel slip models are among the best-performing models in the study ($+91\%$ and $+95\%$ reward relative to the RNN baseline), but at a weight of 1, they are the worst-performing models. One possible explanation is that at higher weights, the auxiliary loss dominates the PPO loss, and the agent learns to predict the privileged information at the expense of the main task.

The auxiliary losses also affect the two architectures very differently. Relative to its own baseline, 26 of the 33 recurrent configurations improve the reward per timestep, compared to only 4 of the 33 feedforward configurations. Of those 4, only the close-ahead curvature loss at a weight of 1.0 is a significant improvement. The same target often has an opposite effect on the two architectures; for example, the forward velocity loss reduces the reward for every weight for the feedforward model, but increases it at every weight for the recurrent model. This suggests that the auxiliary losses mainly

compensate for difficulties in training the recurrent model, rather than improving agents in general.

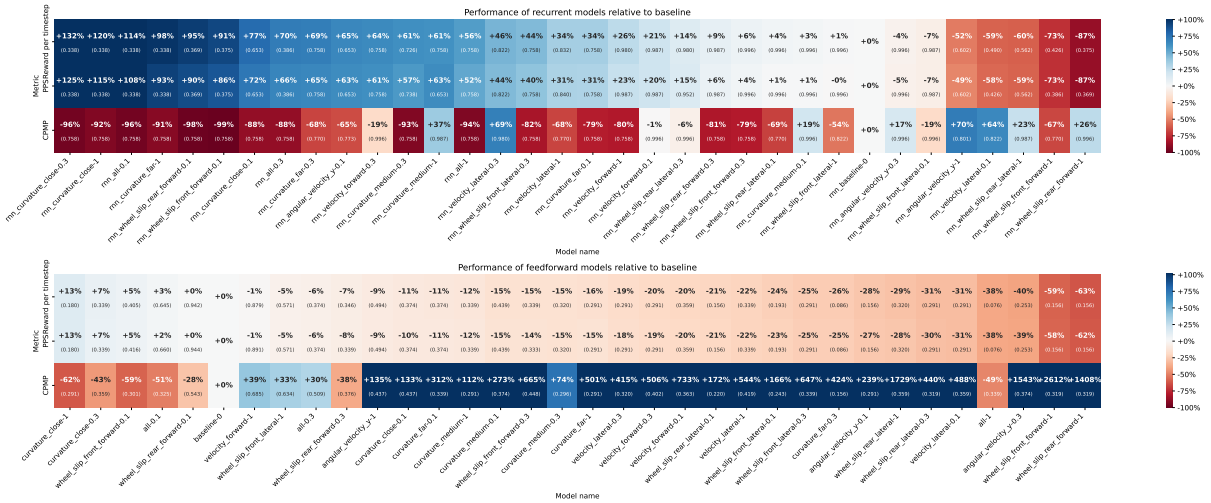


Figure 6.4: Mean of the last 50 training iterations of each model's metrics (mean reward, progress per second, and CPMP) vs the baseline model. The model name consists of the active auxiliary loss group and weight. The p-value is reported in brackets beneath the value in each cell, using a two-sided t-test for the null hypothesis that the means is equal to the baseline model's mean. Corrected for multiple comparisons in each pane using the Benjamini-Hochberg procedure [81]

The findings in Figure 6.4 do not meet the significance standard imposed ($\alpha = 0.05$), so the results should be interpreted with caution. However, the trends are consistent across multiple metrics and are supported by the AUC results in Figure 6.3, which are based on 600 iterations of training rather than just the last 50. Furthermore, the top-performing configurations are consistent across both model architectures.

Figure 6.5 places each model in a 2D space of the mean PPS vs the mean CPMP. The close-ahead curvature auxiliary loss is clearly the most effective in improving both speed and safety, being effective on both model architectures, and across both high and medium weights. Forward wheel slip at a low weight also improves safety. For the feedforward models, the front and rear forward wheel slip losses at a weight of 0.1 reduce collisions per metre of progress by 59% and 28% respectively, while matching or slightly exceeding the baseline's speed. Lateral wheel slip has the opposite effect: the feedforward rear lateral wheel slip models collide roughly 5 to 18 times as often per metre of progress as the baseline.

Collisions per episode should be interpreted with care, since a model that makes little progress also has few opportunities to collide. For example, the recurrent rear forward wheel slip model at a weight of 1 has 77% fewer collisions per episode than the RNN baseline, but also 87% less progress per second. Collisions per metre of progress is therefore the more meaningful measure of safety. Similarly, the very large percentage increases in collisions for some feedforward models partly reflect the low collision rate of the feedforward baseline, which makes small absolute increases appear large in relative terms.

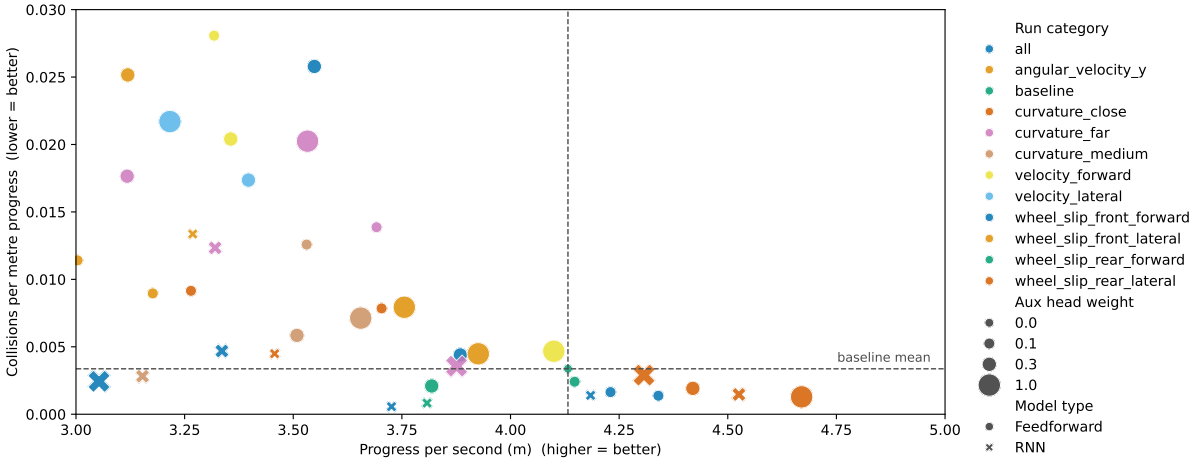


Figure 6.5: Comparison of different models' mean progress per second vs collisions per metre of progress. Bottom right quadrant = faster and safer than the baseline. Bottom left quadrant = slower but safer than the baseline.

Importantly, Figure 6.4 and Figure 6.5 both indicate that the majority of auxiliary losses *do not* improve the feedforward agent's driving capability, and in most cases degrade it. In contrast, the recurrent agent benefits from many auxiliary losses, and in particular from close-ahead curvature and forward wheel slip. This suggests that the recurrent architecture is more difficult to train, and that auxiliary losses can help it learn better representations of the privileged information.

The notable exception from this trend is the close-ahead curvature loss, which improves both architectures' performance, especially at higher weights. This suggests that the close-ahead curvatures are particularly important quantities for the agent to learn in order to perform well.

6.4. Learned Features After Auxiliary Learning

Applying the same probing technique after auxiliary training, we find that the decodability of privileged quantities improved significantly in configurations that performed well for the FF model, but that an increase in decodability does not always translate to an improvement in performance. The full results are visualised in Appendix J, since they are too numerous to include in the main text.

The close-ahead ($\leq 3m$) track curvature auxiliary losses were extremely effective in improving the decodability of the trunk's final layer. In both the feedforward and recurrent models, these losses improved the out-of-sample R^2 scores in close-ahead curvature quantities by between +10 to +49 percentage points, depending on the weight and distance, and even significantly improved the decodability of quantities that were not directly supervised, such as 3.5-m-ahead curvature (up to +27 percentage points). The medium-ahead ($3m < x \leq 6m$) curvature auxiliary loss also improved the decodability of the trunk's final layer, but to a lesser extent.

When losses were introduced for the wheel slip quantities, the decodability of the corresponding quantities improved significantly for smaller loss weights ($w_{\text{aux}} \in \{0.1, 0.3\}$). Interestingly, when forward wheel slip was supervised for the front wheels, the decodability of the back wheel slip also increased. We hypothesise that this is because the car is driven with an all-wheel-drive configuration, so the front and back wheels are coupled, but this was not tested.

Lateral slip supervision had mixed effects, with the decodability of the corresponding quantities improving significantly only for a larger loss weight ($w_{\text{aux}} = 1$), and also resulted in no improvement in the agent's performance.

When all auxiliary losses were enabled, the decodability of almost all quantities improved significantly, but the agent's performance only improved in terms of safety with a low auxiliary loss weight ($w_{\text{aux}} = 0.1$). This suggests that the auxiliary losses may be competing for the model's capacity when they are all enabled.

Chapter 7

Conclusions and recommendations

7.1. Summary

This project set out to determine whether auxiliary losses that predict privileged information improve the performance, training speed, and internal representations of a LiDAR-only end-to-end racing agent. All five objectives listed in Section 1.2 were met; a custom Unity simulator was developed, a baseline PPO agent was trained (Section 5.3), linear regression probes established what the agent natively encodes (Figure 6.1). Then, auxiliary heads were added to predict privileged state information during training (Chapter 5), and the resulting agents were evaluated against the baseline model in terms of racing speed and safety, while also probing the internal representations of the trained agents (Section 6.4). The results show that auxiliary losses can improve both the agent's performance and its internal representations, but that this is not "free lunch": only certain auxiliary losses improve performance, and only at certain loss weights.

The baseline model already encoded its instantaneous dynamics well at initialisation. In contrast, the decodability of close-ahead and medium-ahead track curvature improved significantly during training with auxiliary heads, and the former had an impressive effect on the agent's driving capability, making it both safer and faster. Far-ahead curvature was not learned well by the model, and did not improve the agent's performance when supervised.

7.2. Future work

The results of this project suggest that more work is needed to determine *a priori* which auxiliary losses will improve the agent's performance, and at what loss weights, but that auxiliary losses can indeed improve both the agent's performance and its internal representations. Future work could explore the following avenues:

- Investigate the effect of auxiliary losses on the agent's performance in more complex environments, such as those with dynamic obstacles or varying track conditions.
- Explore the use of adaptive weighting methods for auxiliary losses, to dynamically adjust the weights during training based on the agent's performance and the difficulty of the auxiliary tasks.
- Investigate the effect of auxiliary losses on the agent's performance in multi-agent scenarios, where multiple agents are racing against each other

Bibliography

[1] World Health Organization, *Global Status Report on Road Safety 2018*. Geneva, Switzerland: World Health Organization, 2018, p. 403. Accessed: Oct. 22, 2026. [Online]. Available: https://www.who.int/publications/i/item/9789241565684

[2] J. Hankey *et al.*, "Identification and evaluation of driver errors: Task c report, driver error taxonomy development," *Project no. Dtfh-61-97-c-00051. Virginia Tech Transportation Institute, Blacksburg, VA*, 1999.

[3] A. L. Medina, S. E. Lee, W. W. Wierwille, and R. J. Hanowski, "Relationship between Infrastructure, Driver Error, and Critical Incidents," *Proceedings of the Human Factors and Ergonomics Society Annual Meeting*, vol. 48, no. 16, pp. 2075–2079, 2004, doi: 10.1177/154193120404801661.

[4] B. Olofsson and L. Nielsen, "Using Crash Databases to Predict Effectiveness of New Autonomous Vehicle Maneuvers for Lane-Departure Injury Reduction," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 6, pp. 3479–3490, 2021, doi: 10.1109/TITS.2020.2983553.

[5] C. J. G. V. Driel and B. V. Arem, "The Impact of a Congestion Assistant on Traffic Flow Efficiency and Safety in Congested Traffic Caused by a Lane Drop," *Journal of Intelligent Transportation Systems*, vol. 14, no. 4, pp. 197–208, 2010, doi: 10.1080/15472450.2010.516226.

[6] H. Igliński and M. Babiak, "Analysis of the Potential of Autonomous Vehicles in Reducing the Emissions of Greenhouse Gases in Road Transport," *Procedia Engineering*, vol. 192, pp. 353–358, 2017, doi: https://doi.org/10.1016/j.proeng.2017.06.061.

[7] J. Chen, S. E. Li, and M. Tomizuka, "Interpretable End-to-end Urban Autonomous Driving with Latent Deep Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/2001.08726

[8] M. Jaderberg *et al.*, "Reinforcement Learning with Unsupervised Auxiliary Tasks," in *International Conference on Learning Representations (ICLR 2017)*, 2017. doi: 10.48550/arXiv.1611.05397.

[9] B. Rafiee, J. Jin, J. Luo, and A. White, "What makes useful auxiliary tasks in reinforcement learning: investigating the effect of the target policy." [Online]. Available: https://arxiv.org/abs/2204.00565

[10] M. O'Kelly, H. Zheng, D. Karthik, and R. Mangharam, "F1TENTH: An Open-source Evaluation Environment for Continuous Control and Reinforcement

Learning," in *NeurIPS 2019 Competition and Demonstration Track*, 2020, pp. 77–89.

[11] B. D. Evans *et al.*, "Unifying F1TENTH Autonomous Racing: Survey, Methods and Benchmarks." [Online]. Available: https://arxiv.org/abs/2402.18558

[12] A. Tampuu, T. Matiisen, M. Semikin, D. Fishman, and N. Muhammad, "A Survey of End-to-End Driving: Architectures and Training Methods," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 4, pp. 1364–1384, Apr. 2022, doi: 10.1109/tnnls.2020.3043505.

[13] E. Yurtsever, J. Lambert, A. Carballo, and K. Takeda, "A Survey of Autonomous Driving: Common Practices and Emerging Technologies," *IEEE Access*, vol. 8, pp. 58443–58469, 2020, doi: 10.1109/access.2020.2983149.

[14] P. R. Wurman *et al.*, "Outracing champion Gran Turismo drivers with deep reinforcement learning," *Nature*, vol. 602, no. 7896, pp. 223–228, Feb. 2022, doi: 10.1038/s41586-021-04357-7.

[15] H. Lee *et al.*, "A Champion-Level Vision-Based Reinforcement Learning Agent for Competitive Racing in Gran Turismo 7," *IEEE Robotics and Automation Letters*, vol. 10, no. 6, pp. 5545–5552, Jun. 2025, doi: 10.1109/lra.2025.3560873.

[16] G. Czechmanowski, J. Węgrzynowski, P. Kicki, and K. Walas, "On learning racing policies with reinforcement learning." [Online]. Available: https://arxiv.org/abs/2504.02420

[17] D. A. Pomerleau, "ALVINN: an autonomous land vehicle in a neural network," in *Proceedings of the 2nd International Conference on Neural Information Processing Systems*, in NIPS'88. Cambridge, MA, USA: MIT Press, 1988, pp. 305–313.

[18] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*, 2nd ed. MIT Press, 2018.

[19] M. Bosello, R. Tse, and G. Pau, "Train in Austria, Race in Montecarlo: Generalized RL for Cross-Track F1tenth LIDAR-Based Races," in *2022 IEEE 19th Annual Consumer Communications & Networking Conference (CCNC)*, 2022, pp. 290–298. doi: 10.1109/CCNC49033.2022.9700730.

[20] A. Srinivasan, "Reinforcement Learning: Advancements, Limitations, and Real-world Applications," *INTERANTIONAL JOURNAL OF SCIENTIFIC RESEARCH IN ENGINEERING AND MANAGEMENT*, vol. 7, p. , Aug. 2023, doi: 10.55041/IJSREM25118.

[21] W. Samek, T. Wiegand, and K.-R. Müller, "Explainable Artificial Intelligence: Understanding, Visualizing and Interpreting Deep Learning Models." [Online]. Available: https://arxiv.org/abs/1708.08296

[22] L. Chi and Y. Mu, "Deep Steering: Learning End-to-End Driving Model from Spatial and Temporal Visual Cues." [Online]. Available: https://arxiv.org/abs/1708.03798

[23] J. Mallon, "From Course to Competition: RoboRacer Students' Journey." [Online]. Available: https://www.grasp.upenn.edu/news/grasp-graduate-students-roboracer/

[24] L. Pinto, M. Andrychowicz, P. Welinder, W. Zaremba, and P. Abbeel, "Asymmetric Actor Critic for Image-Based Robot Learning." [Online]. Available: https://arxiv.org/abs/1710.06542

[25] M. Vasco, T. Seno, K. Kawamoto, K. Subramanian, P. R. Wurman, and P. Stone, "A Super-human Vision-based Reinforcement Learning Agent for Autonomous Racing in Gran Turismo." [Online]. Available: https://arxiv.org/abs/2406.12563

[26] G. Hinton, O. Vinyals, and J. Dean, "Distilling the Knowledge in a Neural Network." [Online]. Available: https://arxiv.org/abs/1503.02531

[27] H. Xu, X. Chen, Y. Lang, and Q. Ren, "Vision-Based Policy Learning for High-Speed Autonomous Racing: A Two-Phase Learning Paradigm," *IEEE Robotics & Automation Magazine*, vol. 33, no. 1, pp. 101–114, 2026, doi: 10.1109/MRA.2025.3639812.

[28] S. Ruder, "An Overview of Multi-Task Learning in Deep Neural Networks." [Online]. Available: https://arxiv.org/abs/1706.05098

[29] A. Y. Ng, D. Harada, and S. J. Russell, "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping," in *Proceedings of the Sixteenth International Conference on Machine Learning*, in ICML '99. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc., 1999, pp. 278–287.

[30] A. Navon, I. Achituve, H. Maron, G. Chechik, and E. Fetaya, "Auxiliary Learning by Implicit Differentiation." [Online]. Available: https://arxiv.org/abs/2007.02693

[31] G. Lample and D. S. Chaplot, "Playing FPS Games with Deep Reinforcement Learning," in *Proceedings of the Thirty-First AAAI Conference on Artificial Intelligence*, AAAI Press, 2017, pp. 2140–2146. [Online]. Available: https://arxiv.org/abs/1609.05521

[32] P. Mirowski *et al.*, "Learning to Navigate in Complex Environments." [Online]. Available: https://arxiv.org/abs/1611.03673

[33] K. Chitta, A. Prakash, B. Jaeger, Z. Yu, K. Renz, and A. Geiger, "TransFuser: Imitation with Transformer-Based Sensor Fusion for Autonomous Driving." [Online]. Available: https://arxiv.org/abs/2205.15997

[34] Unity Technologies, *Unity ML-Agents Toolkit*. (2017). GitHub. Accessed: Oct. 13, 2026. [Online]. Available: https://github.com/Unity-Technologies/ml-agents

[35] A. Juliani *et al.*, "Unity: A General Platform for Intelligent Agents," 2018. [Online]. Available: https://arxiv.org/abs/1809.02627

[36] T. V. Samak, C. V. Samak, and M. Xie, "AutoDRIVE Simulator: A Simulator for Scaled Autonomous Vehicle Research and Education," in *2021 2nd International Conference on Control, Robotics and Intelligent System*, in CCRIS'21.

Qingdao, China: Association for Computing Machinery, 2021, pp. 1–5. doi: 10.1145/3483845.3483846.

[37] Unity Technologies, "Unity." [Online]. Available: https://unity.com/

[38] NVIDIA Corporation, "NVIDIA PhysX SDK." [Online]. Available: https://developer.nvidia.com/physx-sdk

[39] "Customised Usage Example." Accessed: Oct. 14, 2026. [Online]. Available: https://f1tenth-gym.readthedocs.io/en/v1.0.0/customized_usage.html#random-track-generator-beta

[40] C. V. Samak and T. V. Samak, "Autonomy Oriented Digital Twins for Real2Sim2Real Autoware Deployment." [Online]. Available: https://arxiv.org/abs/2402.14739

[41] "Unity Documentation." [Online]. Available: https://docs.unity3d.com/6000.5/Documentation/ScriptReference/WheelFrictionCurve.html

[42] H. B. Pacejka and E. Bakker, "THE MAGIC FORMULA TYRE MODEL," *Vehicle System Dynamics*, vol. 21, no. sup001, pp. 1–18, 1992, doi: 10.1080/00423119208969994.

[43] H. Pacejka, *Tyre and Vehicle Dynamics*. in Automotive Engineering Series. Elsevier Science & Technology Books, 2002. [Online]. Available: https://books.google.co.za/books?id=pQNUAAAAMAAJ

[44] K. Cobbe, O. Klimov, C. Hesse, T. Kim, and J. Schulman, "Quantifying Generalization in Reinforcement Learning," in *International Conference on Machine Learning (ICML)*, 2019, pp. 1282–1289.

[45] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, and P. Abbeel, "Domain Randomization for Transferring Deep Neural Networks from Simulation to the Real World," in *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017.

[46] X. B. Peng, M. Andrychowicz, W. Zaremba, and P. Abbeel, "Sim-to-Real Transfer of Robotic Control with Dynamics Randomization," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[47] A. Johnson, *Clipper2: Polygon Clipping, Offsetting & Triangulation in C++, C# and Delphi*. (2022). GitHub. Accessed: Oct. 08, 2026. [Online]. Available: https://github.com/AngusJohnson/Clipper2

[48] B. R. Vatti, "A generic solution to polygon clipping," *Communications of the ACM*, vol. 35, no. 7, pp. 56–63, Jul. 1992, [Online]. Available: https://dl.acm.org/doi/10.1145/129902.129906

[49] G. W. Leibniz, "Meditatio nova de natura anguli contactus et osculi, horumque usu in practica Mathesi, ad figuras faciliore ac exactiore delineandas," *Acta Eruditorum*, pp. 289–292, Jun. 1686.

[50] D. J. Struik, Ed., *A Source Book in Mathematics, 1200-1800*. Cambridge, MA: Harvard University Press, 1969.

[51] J. Stewart, D. Clegg, and S. Watson, *Calculus: Early Transcendentals*. Cengage Learning, 2020. [Online]. Available: https://books.google.co.za/books?id=BCzLDwAAQBAJ

[52] L. P. Kaelbling, M. L. Littman, and A. R. Cassandra, "Planning and acting in partially observable stochastic domains," *Artificial Intelligence*, vol. 101, no. 1–2, pp. 99–134, 1998.

[53] S. P. Singh, T. Jaakkola, and M. I. Jordan, "Learning Without State-Estimation in Partially Observable Markovian Decision Processes," in *Machine Learning Proceedings 1994*, W. W. Cohen and H. Hirsh, Eds., San Francisco (CA): Morgan Kaufmann, 1994, pp. 284–292. doi: https://doi.org/10.1016/B978-1-55860-335-6.50042-8.

[54] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Machine Learning*, vol. 8, no. 3, pp. 229–256, May 1992, doi: 10.1007/BF00992696.

[55] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy Gradient Methods for Reinforcement Learning with Function Approximation," in *Advances in Neural Information Processing Systems*, S. Solla, T. Leen, and K. Müller, Eds., MIT Press, 1999, p. . [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/1999/file/464d828b85b0bed98e80ade0a5c43b0f-Paper.pdf

[56] P. Santana, "An Introduction to Deep Reinforcement and Imitation Learning." [Online]. Available: https://arxiv.org/abs/2512.08052

[57] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms." [Online]. Available: https://arxiv.org/abs/1707.06347

[58] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-Dimensional Continuous Control Using Generalized Advantage Estimation." [Online]. Available: https://arxiv.org/abs/1506.02438

[59] "PyTorch: an imperative style, high-performance deep learning library," in *Proceedings of the 33rd International Conference on Neural Information Processing Systems*, Red Hook, NY, USA: Curran Associates Inc., 2019.

[60] A. Bou *et al.*, "TorchRL: A data-driven decision-making library for PyTorch." [Online]. Available: https://arxiv.org/abs/2306.00577

[61] K. Cho *et al.*, "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation." [Online]. Available: https://arxiv.org/abs/1406.1078

[62] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997, doi: 10.1162/neco.1997.9.8.1735.

[63] J. Węgrzynowski, G. Czechmanowski, P. Kicki, and K. Walas, "Learning dynamics models for velocity estimation in autonomous racing." [Online]. Available: https://arxiv.org/abs/2408.15610

[64] P. J. Huber, "Robust Estimation of a Location Parameter," *The Annals of Mathematical Statistics*, vol. 35, no. 1, pp. 73–101, Mar. 1964, doi: 10.1214/aoms/1177703732.

[65] J. Terven, D. M. Cordova-Esparza, A. Ramirez-Pedraza, E. A. Chavez-Urbiola, and J. A. Romero-Gonzalez, "Loss Functions and Metrics in Deep Learning." [Online]. Available: https://arxiv.org/abs/2307.02694

[66] J. Bergstra and Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, no. 10, pp. 281–305, 2012, [Online]. Available: https://jmlr.org/papers/v13/bergstra12a.html

[67] J. Snoek, H. Larochelle, and R. P. Adams, "Practical Bayesian Optimization of Machine Learning Algorithms," in *Advances in Neural Information Processing Systems*, 2012, pp. 2951–2959. [Online]. Available: https://arxiv.org/abs/1206.2944

[68] J. S. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization," in *Advances in Neural Information Processing Systems*, 2011. [Online]. Available: https://papers.nips.cc/paper_files/paper/2011/hash/86e8f7ab32cfd12577bc2619bc635690-Abstract.html

[69] L. Li, K. Jamieson, G. DeSalvo, A. Rostamizadeh, and A. Talwalkar, "Hyperband: A Novel Bandit-Based Approach to Hyperparameter Optimization," *Journal of Machine Learning Research*, vol. 18, pp. 1–52, 2018, [Online]. Available: https://arxiv.org/abs/1603.06560

[70] L. Li *et al.*, "A System for Massively Parallel Hyperparameter Tuning," in *Proceedings of Machine Learning and Systems*, 2020, pp. 230–246. [Online]. Available: https://arxiv.org/abs/1810.05934

[71] L. Hertel, P. Baldi, and D. L. Gillen, "Quantity vs. Quality: On Hyperparameter Optimization for Deep Reinforcement Learning." [Online]. Available: https://arxiv.org/abs/2007.14604

[72] Z. Chen, V. Badrinarayanan, C.-Y. Lee, and A. Rabinovich, "GradNorm: Gradient Normalization for Adaptive Loss Balancing in Deep Multitask Networks," Nov. 2017. [Online]. Available: https://arxiv.org/abs/1711.02257

[73] A. Kendall, Y. Gal, and R. Cipolla, "Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, IEEE, Jun. 2018, pp. 7482–7491. [Online]. Available: https://arxiv.org/abs/1705.07115

[74] G. Alain and Y. Bengio, "Understanding intermediate layers using linear classifier probes." [Online]. Available: https://arxiv.org/abs/1610.01644

[75] A. Anand, E. Racah, S. Ozair, Y. Bengio, M.-A. Côté, and R. D. Hjelm, "Unsupervised State Representation Learning in Atari." [Online]. Available: https://arxiv.org/abs/1906.08226

[76] Y. Belinkov, "Probing Classifiers: Promises, Shortcomings, and Advances." [Online]. Available: https://arxiv.org/abs/2102.12452

[77] T. McGrath *et al.*, "Acquisition of chess knowledge in AlphaZero," *Proceedings of the National Academy of Sciences*, vol. 119, no. 47, p. e2206625119, 2022, doi: 10.1073/pnas.2206625119.

[78] W. Gurnee and M. Tegmark, "Language Models Represent Space and Time." [Online]. Available: https://arxiv.org/abs/2310.02207

[79] K. He, X. Zhang, S. Ren, and J. Sun, "Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification." [Online]. Available: https://arxiv.org/abs/1502.01852

[80] T. J. DiCiccio and B. Efron, "Bootstrap confidence intervals," *Statistical Science*, vol. 11, no. 3, pp. 189–228, 1996, doi: 10.1214/ss/1032280214.

[81] Y. Benjamini and Y. Hochberg, "Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing," *Journal of the Royal Statistical Society: Series B (Methodological)*, vol. 57, no. 1, pp. 289–300, 1995, doi: https://doi.org/10.1111/j.2517-6161.1995.tb02031.x.

[82] P. Thomas, "Bias in Natural Actor-Critic Algorithms," in *Proceedings of the 31st International Conference on Machine Learning*, E. P. Xing and T. Jebara, Eds., in Proceedings of Machine Learning Research, vol. 32. Beijing, China: PMLR, 2014, pp. 441–448. [Online]. Available: https://proceedings.mlr.press/v32/thomas14.html

Appendix

Appendix A Project Timeline

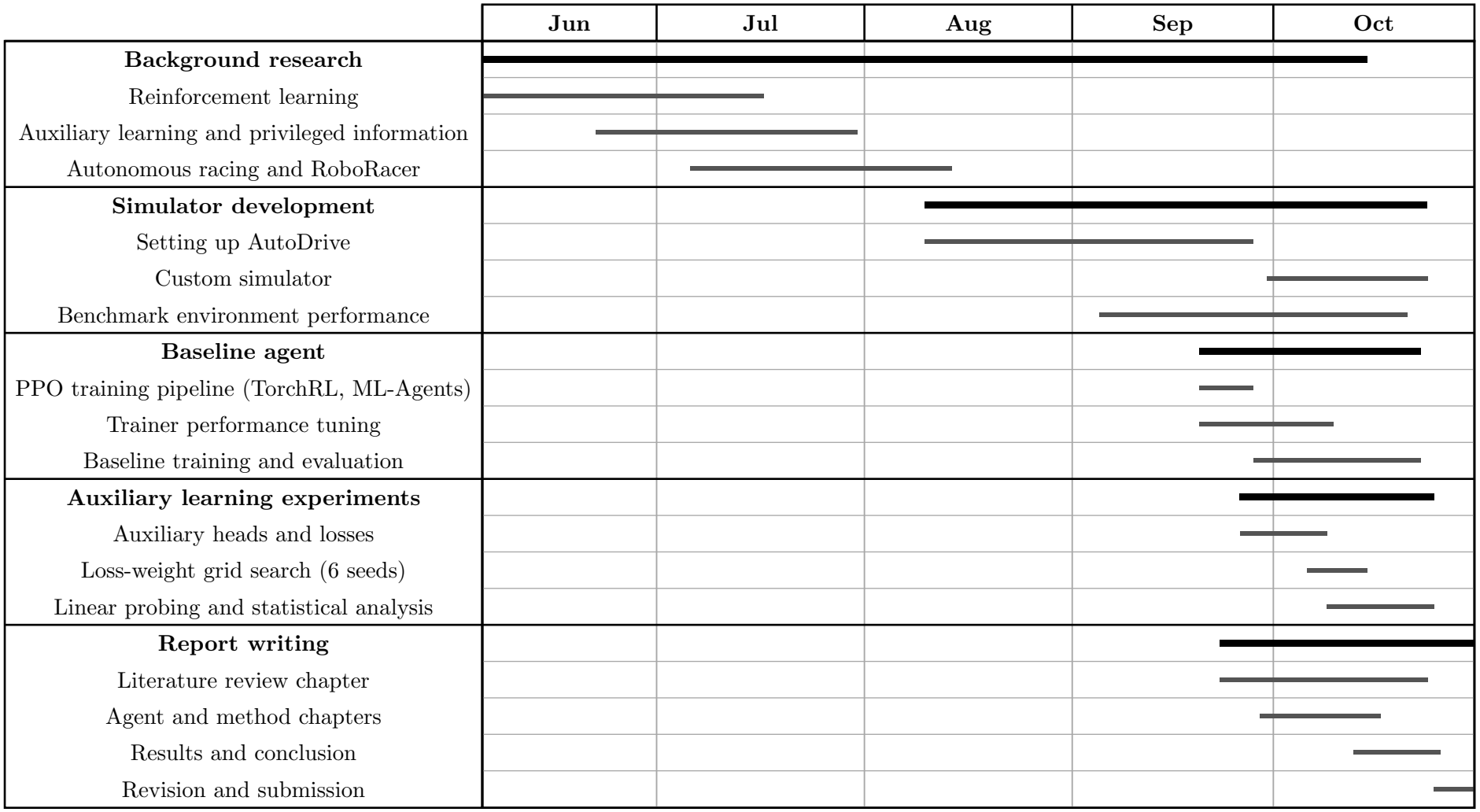


Figure A.1: Timeline of the project, showing the main tasks and their durations.

Appendix B Graduate Attributes

B.1 Complex Engineering Problem

The project addresses a complex engineering problem because it requires a fundamentals-based formulation of autonomous racing as a POMDP and first-principles-based design of vehicle dynamic models. It draws on specialist knowledge of RL and autonomous racing, and engages with current research literature on privileged information, auxiliary learning, and representation probing. The problem has no obvious solution: a LiDAR-only agent must learn quantities it cannot observe, and it is not clear which privileged targets or loss weights would help. It involves conflicting requirements (speed vs. safety), balancing the loss weights of the primary task vs. auxiliary tasks, and considering the trade-offs between simulation fidelity and training speed. It also involves many coupled systems

Appendix C AI Declaration

- Claude was used for searching literature, checking the report for errors, advice on clarity of writing, and changing the format of references from BibTex to Hayagriva. Furthermore, Claude was used to explain various tensor shape bugs in PyTorch code, but the final implementation was always done manually, as well as to explain how TorchRL works. Initially, Claude was used to generate diagrams, but I did not like how little control I had over the diagrams, so re-implemented them manually from scratch. Claude was used to generate parts of Figure 4.5 based on a handmade diagram. Claude was used to explain to me why certain training setups may perform badly on the hardware available. Claude was used to explain how to perform the FDR analysis on Appendix J, but the code was written by me, and I understand the process.
- Google Gemini was used to a lesser extent for searching literature, and, before starting the project, for assistance in understanding RL in conjunction with the David Silver videos. Gemini was instrumental in explaining potential-based reward shaping in an easy-to-understand way. Gemini was used to do spelling & grammar checking.
- Google Scholar Labs was used extensively in searching for literature.
- DeepSeek was used to search for literature, and to provide input on the structure and content of the report. For example, it advised which sections could be reduced in length without sacrificing report quality.

AI was not used to generate the training code or the model definition code - that was hand-written.

Appendix D WheelCollider properties

The values in Table D.1 are the parameters of the WheelCollider model used in the Unity simulator. These parameters were fit by Samak and Samak [40] tothe RoboRacer vehicle, and are used in the simulator to model the vehicle's dynamics. The WheelCollider model is a simplified physics model that approximates the behaviour of a real wheel, and is used in Unity to simulate the vehicle's movement and interaction with the environment. Identical parameters are used for all four wheels of the vehicle.

Table D.1: Relevant parameters of the WheelCollider model. * = forward & sideways

Property	Description	Value for RoboRacer
`radius`	Radius of the wheel	5.8 cm
`mass`	Mass of each wheel; determines its rotational inertia	109 g
`suspension_distance`	Maximum travel of the suspension along the wheel's local Y axis	5 cm
`suspension_spring`	Spring stiffness pulling the wheel toward its target position	500 N/m
`suspension_damper`	Damping that resists the speed of suspension movement	100 (N · s/m)
`extremum_slip`*	Slip at which the friction curve peaks (maximum grip)	0.15 fwd / 0.01 side
`extremum_value`*	Friction at the peak of the curve	0.9 fwd / 1 side
`asymptote_slip`*	Slip beyond which the tyre is fully sliding	0.25 fwd / 0.1 side
`asymptote_value`*	Friction while sliding	0.58 fwd / 0.5 side
`stiffness`*	Multiplier applied to the whole friction curve	0.8 fwd / 1 side

Appendix E Derivation of the Policy Gradient Theorem

This appendix provides a proof of the policy gradient theorem, where, given a policy π_θ parameterised by θ, the gradient of the expected return $J(\theta)$ with respect to θ can be expressed as an expectation over trajectories τ sampled from the policy:

$$J(\theta) = \mathbb{E}_{\tau\sim p_\theta}[G_0] = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T}\gamma^t r_t\right] = \int p_\theta(\tau)G(\tau)\,\mathrm{d}\tau$$

Using the *log-derivative* trick, $\nabla_\theta p_\theta(\tau) = p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)$, and assuming that the integral and derivative can be interchanged[8], the gradient of the expected return with respect to θ is then

$$\begin{aligned}\nabla_\theta J(\theta) = \nabla_\theta \int p_\theta(\tau)G(\tau)\,\mathrm{d}\tau = \int \nabla_\theta p_\theta(\tau)G(\tau)\,\mathrm{d}\tau = \int p_\theta(\tau)\nabla_\theta \log p_\theta(\tau)G(\tau)\,\mathrm{d}\tau \\ = \mathbb{E}_{\tau\sim p_\theta}[\nabla_\theta \log p_\theta(\tau)G(\tau)]\end{aligned}$$

Considering that the probability of a trajectory can be factorised as

$$p_\theta(\tau) = p(s_0)\prod_{t=0}^{T-1}\underbrace{p(o_t \mid s_t)}_{\mathcal{Z}}\cdot\pi_\theta(a_t \mid o_t)\cdot\underbrace{p(r_t \mid s_t, a_t, s_{t+1}, o_t)}_{\mathcal{R}}\cdot\underbrace{p(s_{t+1} \mid s_t, a_t)}_{\mathcal{P}}$$

and since $p(s_0)$, $\mathcal{P}$, $\mathcal{Z}$ and $\mathcal{R}$ do not depend on θ, their factors vanish, so

$$\nabla_\theta \log p_\theta(\tau) = \sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(a_t \mid o_t)$$

Then, the *policy gradient* can be expressed as

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(a_t \mid o_t)G(\tau)\right]$$

Write $G(\tau) = G_0$. Splitting the return at time t,

$$G_0 = \sum_{k=0}^{t-1}\gamma^k r_k + \gamma^t G_t, \quad G_t = \sum_{k=t}^{T-1}\gamma^{k-t} r_k,$$

where G_t is the discounted *reward-to-go*,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau\sim p_\theta}\left[\sum_{t=0}^{T-1}\nabla_\theta \log \pi_\theta(a_t \mid o_t)\left(\sum_{k=0}^{t-1}\gamma^k r_k + \gamma^t G_t\right)\right]$$

However, since the expectation of the fixed past-reward terms is zero, they can be removed from the policy gradient without introducing bias. This can be seen by conditioning on the history $h_t = (s_0, o_0, a_0, r_0, ..., s_{t-1}, o_{t-1}, a_{t-1}, r_{t-1}, s_t, o_t)$:

[8] This can be done only under certain regularity conditions not explored in this report, but generally if the functions are continuous and differentiable everywhere, and the integral is finite.

$$\mathbb{E}_{a_t \sim \pi_\theta}[\nabla_\theta \log \pi_\theta(a_t \mid o_t) \mid h_t] = \int_a \nabla_\theta \pi_\theta(a \mid o_t) = \nabla_\theta \int_a \pi_\theta(a \mid o_t) = \nabla_\theta 1 = 0,$$

The past-reward terms therefore vanish in expectation, leaving

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim p_\theta}\left[\sum_{t=0}^{T-1} \gamma^t \nabla_\theta \log \pi_\theta(a_t \mid o_t) G_t\right].$$

Removing these zero-mean terms typically reduces the variance of the Monte Carlo estimate. In practice the factor γ^t is usually dropped. This yields a biased estimate of $\nabla_\theta J(\theta)$ for $\gamma < 1$ [82] however.

Appendix F Distribution of Auxiliary Targets

Table F.1 shows the proportion of auxiliary target values within 1 standard deviation of the mean, and the proportion of values beyond 3 standard deviations of the mean. It is used in Section 5.3.3 to justify the choice of the Huber loss function for the auxiliary losses, since it is more robust to outliers than the SE loss.

Table F.1: Proportion of auxiliary target values within 1 standard deviation of the mean, and proportion of values beyond 3 standard deviations of the mean. The last column shows the ratio of the proportion of values beyond 3 standard deviations to the expected proportion for a normal distribution (0.27%).

Auxiliary target	$\hat{P}(\lvert X\rvert < 1\sigma)$	$\hat{P}(\lvert X\rvert > 3\sigma)$	$\hat{P}(\lvert X\rvert > 3\sigma) \div P_{\text{normal}}(\lvert X\rvert > 3\sigma)$
velocity_forward	90.72%	4.74%	17.54x
velocity_lateral	64.62%	0%	0.01x
angular_velocity_y	64.38%	0%	0x
track_curvature_0_50	76.27%	1.36%	5.02x
track_curvature_1_00	76.22%	1.51%	5.6x
track_curvature_1_50	79.35%	2.11%	7.8x
track_curvature_2_00	77.39%	1.37%	5.06x
track_curvature_2_50	77.55%	1.64%	6.08x
track_curvature_3_00	77.75%	1.66%	6.16x
track_curvature_3_50	77.68%	1.7%	6.28x
track_curvature_4_00	76.78%	1.51%	5.61x
track_curvature_4_50	76.06%	1.56%	5.77x
track_curvature_5_00	77.15%	1.55%	5.76x
track_curvature_5_50	76.85%	1.42%	5.25x
track_curvature_6_00	76.72%	1.54%	5.7x
track_curvature_6_50	76.76%	1.54%	5.7x
track_curvature_7_00	76.96%	1.57%	5.8x
track_curvature_7_50	77.79%	1.63%	6.05x
track_curvature_8_00	77.31%	1.59%	5.9x
wheel_slip_forward_front_left	94.29%	2.91%	10.79x
wheel_slip_forward_front_right	93.97%	2.98%	11.03x
wheel_slip_lateral_front_left	75.26%	1%	3.72x
wheel_slip_lateral_front_right	73.96%	0.72%	2.66x
wheel_slip_forward_back_left	94.19%	2.92%	10.81x
wheel_slip_forward_back_right	93.87%	2.99%	11.09x
wheel_slip_lateral_back_left	74.11%	1.05%	3.91x
wheel_slip_lateral_back_right	73.47%	1.05%	3.88x

Appendix G Baseline R^2 Scores

The below table shows the out-of-sample R^2 scores of linear regression probing of the final layer of the trunk on the recurrent and feedforward baseline models. The scores are reported at initialisation and after 600 PPO iterations. These results are used in Section 6.2 to establish what the baseline models' internal representations encode without auxiliary supervision, and what the effect of training is on the decodability of privileged quantities.

Table G.1: Out-of-sample R^2 scores for linear regression probing of the final layer of the trunk on the recurrent and feedforward baseline models. R^2 scores are reported at initialisation and after 600 PPO iterations.

	Feedforward baseline		Recurrent baseline	
Auxiliary target	Initialisation	Trained	Initialisation	Trained
velocity_forward	52.2%	62.4%	92.9%	75.6%
velocity_lateral	95.5%	92.5%	95.6%	90.6%
angular_velocity_y	95.9%	91.8%	95.7%	89.7%
track_curvature_0_50	25%	31.9%	29.5%	22.8%
track_curvature_1_00	29.1%	40.1%	38.1%	32.9%
track_curvature_1_50	17%	34.2%	25.2%	28.8%
track_curvature_2_00	8.4%	29.7%	10.4%	23.7%
track_curvature_2_50	6.9%	25.5%	3.7%	22.5%
track_curvature_3_00	5.9%	18.1%	2%	18.1%
track_curvature_3_50	6.7%	15%	3.3%	18.2%
track_curvature_4_00	6.5%	12.5%	3.7%	15.9%
track_curvature_4_50	6.6%	11.5%	3.5%	13.4%
track_curvature_5_00	6.2%	10.1%	3.6%	11.1%
track_curvature_5_50	5.4%	8.7%	3.4%	9.2%
track_curvature_6_00	6%	9.5%	5.4%	10.1%
track_curvature_6_50	6.3%	8.8%	5.7%	9.8%
track_curvature_7_00	6.9%	9.5%	6.2%	10.3%
track_curvature_7_50	6.6%	9.4%	6.6%	10.5%
track_curvature_8_00	6.5%	8.8%	6.5%	9.4%
wheel_slip_forward_front_left	18.2%	31.2%	80.3%	53.9%
wheel_slip_forward_front_right	19.7%	27.5%	80%	51.5%
wheel_slip_lateral_front_left	88.4%	86.9%	86.5%	79.8%
wheel_slip_lateral_front_right	87.8%	86.5%	86.3%	80.6%
wheel_slip_forward_back_left	18.9%	31.3%	80.4%	53.4%
wheel_slip_forward_back_right	20.6%	27.6%	80.4%	51.4%
wheel_slip_lateral_back_left	83.6%	79.2%	86.2%	79.2%
wheel_slip_lateral_back_right	82.9%	78.4%	85.6%	78.9%

Appendix H Mean Reward Per Timestep of a Baseline FF Model

The below figure shows the mean reward per timestep of a test baseline feedforward model that was executed before the hyperparameter grid search. We use this figure in Chapter 6 to justify the choice of 600 training iterations for each configuration, since it converges after 400-600 iterations.

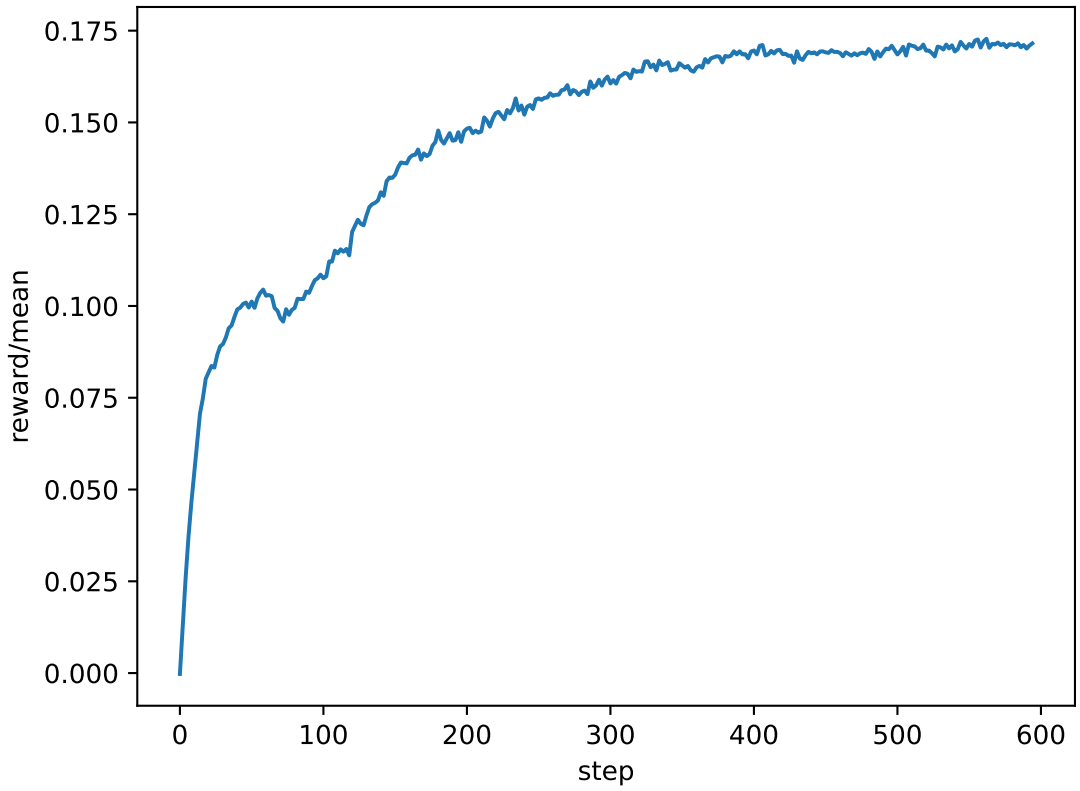


Figure H.1: Mean reward per timestep of a baseline feedforward model over 600 training iterations, averaged across the timesteps in each iteration. Qualitatvely, the model converges after 400-600 iterations

Appendix I Training Performance Benchmark Results

Table I.1 shows the benchmark results of training performance with various number of concurrent trainers, trainer threads, and parallel environments. Trainer threads indicates the number of parallel PyTorch processes per trainer.

Table I.1: Training performance results, sorted by throughput.

Concurrent trainers	Trainer threads	Parallel envs[9]	Throughput (timesteps/sec)	Speedup vs realtime[10]
8	16	8	4172.37	166.89x
8	16	4	4020.17	160.81x
8	16	8	3999.46	159.98x
8	16	10	3962.12	158.48x
12	8	4	3853.04	154.12x
8	16	8	3843.59	153.74x
12	16	10	3805.19	152.21x
8	8	4	3802.34	152.09x
12	8	8	3751.41	150.06x
12	8	10	3717.67	148.71x
12	16	8	3582.39	143.3x
4	16	10	3428.19	137.13x
12	16	4	3302.99	132.12x
8	8	8	3207.98	128.32x
4	16	8	3201.63	128.07x
8	8	10	3164.72	126.59x
4	16	4	2684.4	107.38x
4	8	4	2414.64	96.59x
2	16	10	2206.84	88.27x
4	8	16	2086.99	83.48x
4	16	16	2075.62	83.02x
2	16	8	2034.77	81.39x
4	8	10	1884.89	75.4x
4	8	8	1830.29	73.21x
2	16	4	1557.77	62.31x
2	8	4	1385.09	55.4x
2	8	16	1122.47	44.9x
2	16	16	1122.1	44.88x
2	8	10	1013.7	40.55x
2	8	8	977.93	39.12x

[9]It was not possible to run 16 parallel envs on 12 trainers, because the machine ran out of memory.
[10]Assuming 25 Hz decision frequency

Appendix J R^2 Scores After Training with Auxiliary Heads

The following two figures, Figure J.1 and Figure J.2, show the difference in mean out-of-sample R^2 scores of linear regression probing of the final layer of the trunk on the feedforward and recurrent models, respectively, after training with auxiliary heads. The X-axis shows the auxiliary target, and the Y-axis shows the auxiliary training configuration. Red boxes indicate the features that are supervised by the auxiliary loss. Bold where the null hypothesis that the means are equal is rejected ($\alpha = 0.1$), corrected for false discovery rate using the Benjamini-Hochberg procedure [81].

These figures show that the decodability of privileged quantities is improved significantly in the close- and medium-ahead curvature auxiliary loss configurations, as well as when all auxiliary losses are enabled.

In most configurations, the decodability of the forward wheel slip quantities decreases when auxiliary heads not related to forward wheel slip are enabled.

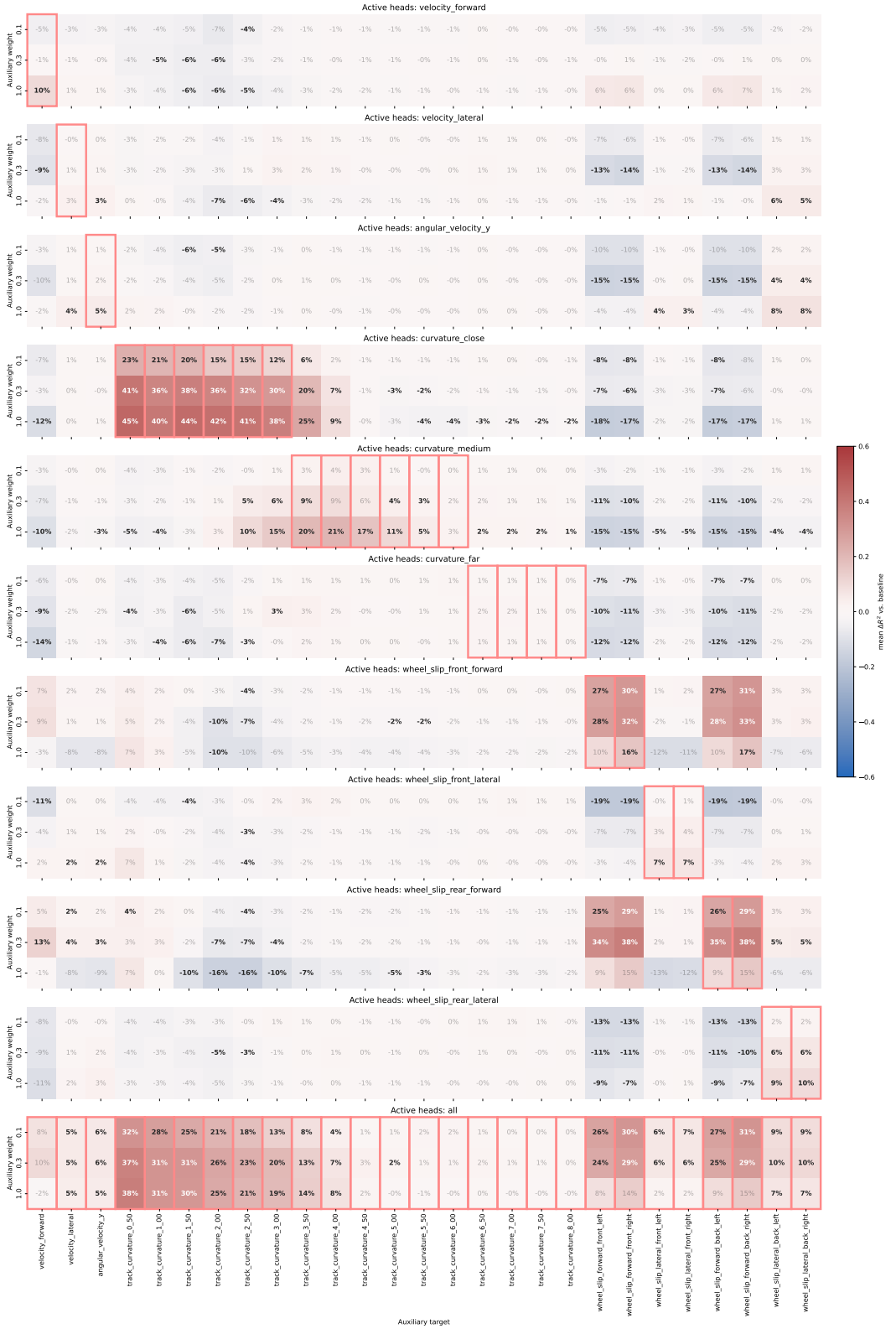


Figure J.1: Difference in mean out-of-sample R^2 vs the baseline feedforward PPO model. The X-axis shows the auxiliary target, and the Y-axis shows the auxiliary training configuration. Red boxes indicate the features that are supervised by the auxiliary loss. Bold where the null hypothesis that the means are equal is rejected ($\alpha = 0.1$), corrected for false discovery rate using the Benjamini-Hochberg procedure [81]

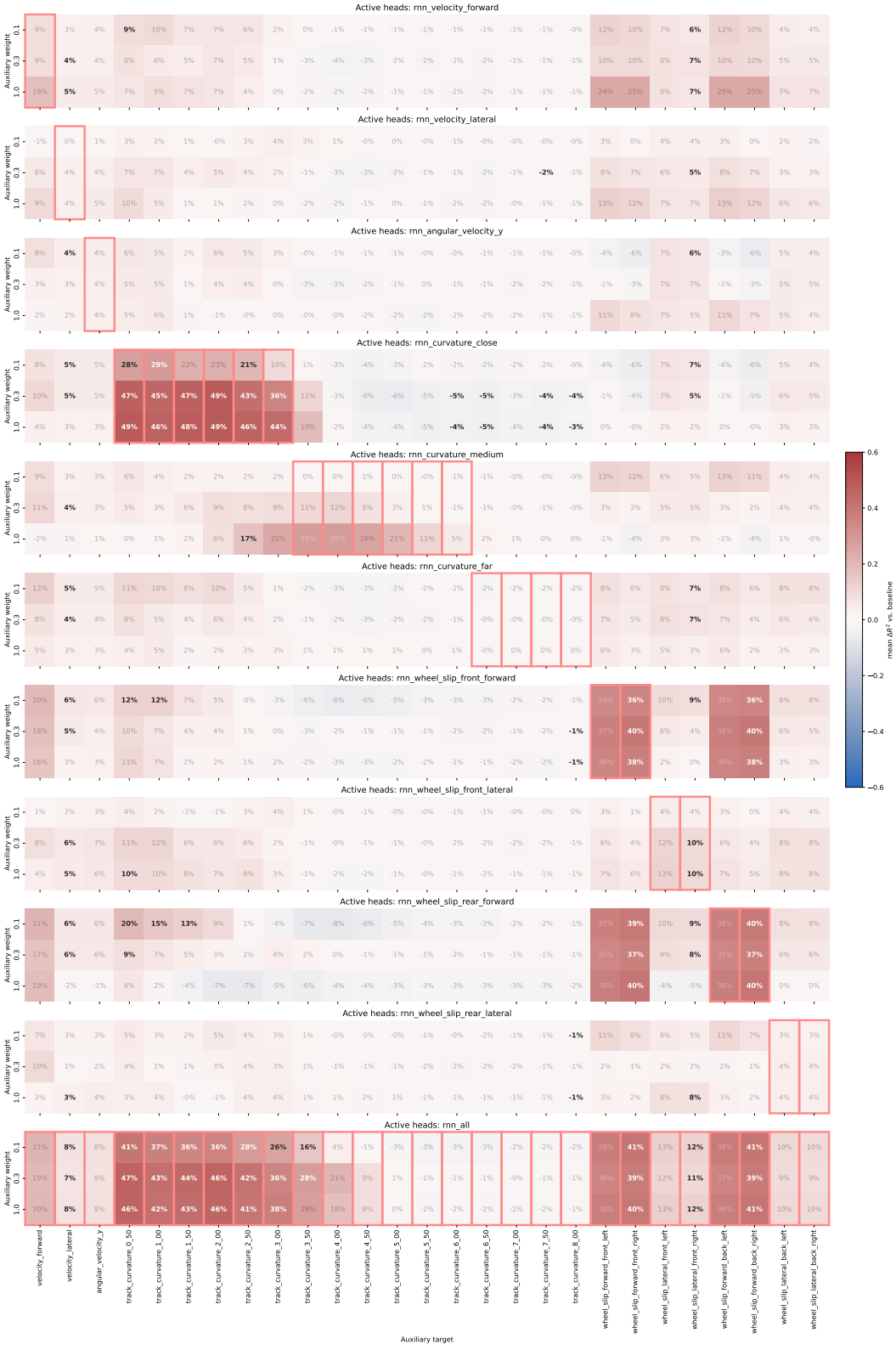


Figure J.2: Difference in mean out-of-sample R^2 vs the baseline recurrent PPO model. The X-axis shows the auxiliary target, and the Y-axis shows the auxiliary training configuration. Red boxes indicate the features that are supervised by the auxiliary loss. Bold where the null hypothesis that the means are equal is rejected ($\alpha = 0.1$), corrected for false discovery rate using the Benjamini-Hochberg procedure [81]